20AD503 MACHINE LEARNING TECHNIQUES L T P C

3 0 0 3

OBJECTIVES:

The Student should be made to:

- Understand the basics of machine learning (ML)
- Understand the methods of machine learning
- Know about the implementation aspects of machine learning
- Understand the concepts of data analytics and machine learning
- Understand and implement use cases of ML

UNIT I MACHINE LEARNING BASICS 9

Introduction to Machine Learning (ML) - Essential concepts of ML - Types of learning - Machine learning methods based on Time - Dimensionality - Linearity and Non linearity - Early trends in Machine learning - Data Understanding Representation and visualization.

UNIT II MACHINE LEARNING METHODS 9

Linear methods - Regression - Classification - Perceptron and Neural networks - Decision trees - Support vector machines - Probabilistic models - Unsupervised learning - Featurization.

UNIT III MACHINE LEARNING IN PRACTICE 9

Ranking - Recommendation System - Designing and Tuning model pipelines - Performance measurement - Azure Machine Learning - Open-source Machine Learning libraries - Amazon's Machine Learning Tool Kit: Sagemaker.

UNIT IV MACHINE LEARNING AND DATA ANALYTICS 9

Machine Learning for Predictive Data Analytics - Data to Insights to Decisions - Data Exploration - Information based Learning - Similarity based learning - Probability based learning - Error based learning - Evaluation - The art of Machine learning to Predictive Data Analytics.

UNIT V APPLICATIONS OF MACHINE LEARNING 9

Image Recognition - Speech Recognition - Email spam and Malware Filtering - Online fraud detection - Medical Diagnosis.

TOTAL: 45 PERIODS

OUTCOMES:

On successful completion of this course, the students will be able to,

- Study and use the basics of machine learning
- Explain various ZMachine Learning methods
- Demonstrate various ML techniques using standard packages
- Explore knowledge on Machine learning and Data Analytics
- Apply ML to various real time examples

TEXT BOOKS:

1. Ameet V Joshi, “Machine Learning and Artificial Intelligence”, Springer Publications, 2020.
2. John D Kelleher , Brain Mac Namee, Aoife D'Arey, “Fundamentals of Machine learning for Predictive Data Analytics, Algorithms, Worked Examples and case studies”, MIT press, 2015.

REFERENCES:

1. Christopher M Bishop, “Pattern Recognition and Machine Learning”, Springer Publications, 2011.
2. Stuart Jonathan Russell, Peter Norvig, John Canny, “Artificial Intelligence: A Modern Approach”, Prentice Hall, 2020.
3. John Paul Muller, Luca Massaron, “Machine Learning Dummies” Wiley Publications, 2021.

UNIT I

MACHINE LEARNING BASICS

Introduction to Machine Learning (ML) - Essential concepts of ML - Types and Non linearity -Early trends in Machine learning - Data Understanding Representation and visualization.

INTRODUCTION TO AI AND ML

We are not going to try and define these entities with word play, but rather are going to learn the context around the origins and scope of these entities. This will make their meaning apparent. The roots of these words originate from multiple disciplines and not just computer science.

In spite of being such multi-disciplinary, there are several things that we can conceptually learn collectively and obtain clarity of its scope. That is the primary objective behind this book. I want to make the field accessible to newcomers to the field in the form of graduate level students to engineers and professionals who are entering the field or even actively working in this field.

WHAT IS ML

The term “Machine Learning” or ML in short, was coined in 1959 by Arthur Samuel in the context of solving game of checkers by machine. The term refers to a computer program that can learn to produce a behavior that is not explicitly programmed by the author of the program. Rather it is capable of showing behavior that the author may be completely unaware of.

ESSENTIAL CONCEPTS IN ARTIFICIAL INTELLIGENCEAND MACHINE LEARNING

Big Data and Not-So-Big Data

What Is Big Data

Big data processing has become a very interesting and exciting chapter in the field of AI and ML since the advent of cloud computing. Although there is no specific memory size above which the data is called big, the generally accepted definition is as follows:

When the size of data is large enough such that it cannot be processed on a single machine,it is called as big data. Based on current (2018) generation of computers this equates to something roughly more than 10 GB. From there it can go to hundreds of petabytes (1 petabyte is 1000 terabytes and 1 terabyte is 1000 gigabytes) and more.

TYPES OF LEARNING

The machine learning algorithms are broadly classified into three types:

1 Supervised learning algorithms

2 Unsupervised learning algorithms

3 Reinforcement learning algorithmsLet's look at each type in detail.

Supervised Learning

For simplicity, let's consider the ML system as a black box that produces some output when given some input. If we already have a historical data that contains the set of outputs for a set of inputs, then the learning based on this data is called as supervised learning. A classic example of supervised learning is classification.

Unsupervised Learning

In unsupervised learning paradigm, the labelled data is not available. A classic example of unsupervised learning is "clustering." Consider the same example as described in the previous subsection, where we have measurements of the sepal and petal dimensions of three types of flowers. However, in this case, we don't have exact names of the flowers for each set of measurements.

Reinforcement Learning

Reinforcement learning is a special type of learning method that needs to be treated separately from supervised and unsupervised methods. Reinforcement learning involves feedback from the environment, hence it is not exactly unsupervised, however, it does not have a set of labelled samples available for training as well and hence it cannot be treated as supervised. In reinforcement learning methods, the system is continuously interacting with the environment in search of producing desired behavior and is getting feedback from the environment.

MACHINE LEARNING METHODS BASED ON TIME

Another way to slice the machine learning methods is to classify them based on the type of data that they deal with. The systems that take in static labelled data are called static learning methods. The systems that deal with data that is continuouslychanging

with time are called dynamic methods. Each type of methods can be supervised, or unsupervised, however, reinforcement learning methods are always dynamic.

Static Learning

Static learning refers to learning on the data that is taken as a single snapshot and the properties of the data remain constant over time. Once the model is trained on the data (either using supervised or unsupervised learning) the trained model can be applied to similar data anytime in the future and the model will still be valid and will perform as expected. Typical examples of this would be image classification of different animals.

Dynamic Learning

This is also called as time series based learning. The data in this type of problems is time sensitive and keeps changing over time. Hence the model training is not a static process, but the model needs to be trained continuously (or after every reasonable time window) to remain effective. A typical example of such problems is weather forecasting, or stock market predictions. A model trained a year back will be completely useless to predict the weather for tomorrow, or predict the priceof any stock for tomorrow. The fundamental difference in the two types is notion of state. In static models, the state of the model never changes, while in dynamic models the state of the model is a function oftime and it keeps changing.

DIMENSIONALITY

In order to understand these high dimensions, we need to look at the fundamental property of dimensions. The space dimensions are defined such that each of the dimensions is perpendicular or orthogonal to other two. This property of orthogonality is essential to have unique representation of all the points in this 3-dimensional space. If the dimensions are not orthogonal to each other, then one can have multiple representations for same points in the space and whole mathematical calculations based on it will fail.

For example if we setup the three coordinates as length, width, and height with some arbitrary origin (The precise location of origin only changes the value of coordinates, but does not affect the uniqueness property and hence any choice of origin is fine as long as it remains unchanged throughout the calculation.) The coordinates (0,0,0) mark the location of origin itself. The coordinates (1,1,1) will mark a point space that is 1 unit away from the origin in each of the dimensions and is unique. No other set of coordinates will mean the same location in space.

1.8.1 Curse of Dimensionality

Even if adding arbitrarily large number of dimensions is fine from mathematical standpoint, there is one problem. With increase in dimensions the density of the data gets diminished exponentially. For example if we have 1000 data points in training data, and data has 3 unique features. Let's say the value of all the features is within 1–10. Thus all these 1000 data points lie in a cube of size $10 \times 10 \times 10$.

LINEARITY AND NONLINEARITY

Concept of linearity and nonlinearity is applicable to both the data and the model that built on top of it. However, the concept of linearity differs in each context. Data is called as linear if the relationship between the input and output is linear. OCCAM'S RAZOR

In developing and applying machine learning models one always comes across multiple possible solutions and multiple possible approaches to get the answer. Many times there is no theoretical guideline as to which solution or which approach is better than the rest. In such cases the concept of *Occam's Razor*, which is also sometimes called as *principle of parsimony* can be effectively applied. The principle states,

Definition 1.10 Occam's Razor One should not make more assumptions than the minimum needed, or in other words, when there are multiple alternatives for a solution, the simplest approach is the best.

This principle is not quite a theorem and cannot be applied as a quantitative rule orequation. However, it stands a strong effective conceptual guideline when making such decisions in real life. It is also important to note that this rule creates a form of tradeoff, where on one side we have more information in the form of more complexity and on the other hand less information in the form of simplicity. One should not oversimplify the problem such that some of the core information is lost.Another derived aspect of Occam's razor is the simpler solutions tend to have more generalization capabilities.

No Free Lunch Theorem

Another interesting concept that is good to be aware of when designing a machine learning system comes from a paper by Wolpert and Macready [59], in the formof no free lunch theorem or NFL theorem in optimization. The theorem essentiallystates that

Definition 1.10.1 NFL Theorem If an algorithm performs better on certain class of problems, then it pays for it in the form of degraded performance in other classes of problems. In other words, you cannot have single optimal solution for all classes of

problems.

Law of Diminishing Returns

The law of diminishing returns is typically encountered in economics and business scenarios, where it states that adding more headcounts to complete a job starts to yield less and less returns with increase in existing number of headcount [9].

Early Trends in Machine Learning

Before the machine learning started off commercially in true sense, there were fewother systems that were already pushing the boundary if routine computation. One such notable application was *Expert Systems*.

Expert Systems

The definition by Alan Turin marks the beginning of the era where machine intelligence was recognized and with that field of AI was born.

Definition 1.10.4: Expert Systems An intelligent computer program that uses knowledge and inference procedures to solve problems that are difficult enough to require significant human expertise for the solution .

DATA UNDERSTANDING, REPRESENTATION, ANDVISUALIZATION

Before starting with the theory of machine learning in the next part of the book, this chapter focusses on the understanding, representing, and visualizing the data. We will use the techniques described here multiple times throughout the book. These steps together can be called as data preprocessing.

With recent explosion of small devices that are connected to internet,[1] the amount of data that is being generated has increased exponentially. This data can be quite useful for generating variety of insights if handled properly, else it can only burden the systems handling it and slow down everything. The science that deals with general handling and organizing and then interpreting the data is called as data science. This is a fairly generic term and concepts of machine learning and artificial intelligence are part of it.

Understanding the Data

First step in building an application of artificial intelligence is to understand the data. The data in raw form can come from different sources, in different formats. Some data can be missing, some data can be mal-formatted, etc. It is the first task to get familiar with the data. Clean up the data as necessary.

The step of understanding the data can be broken down into three parts:

1. Understanding entities
2. Understanding attributes
3. Understanding data types

In order to understand these concepts, let's us consider a data set called *Iris data* [3]. Iris data is one of the most widely used data set in the field of machine learningfor its simplicity and its ability to illustrate many different aspects of machine learning atone place. Specifically, Iris data states a problem of multi-class classification of three different types of flowers, *Setosa*, *Versicolor*, and *Virginica*. The data set is ideal for learning basic machine learning application as it does not contain any missing values, and all the data is numerical. There are 4 features per sample, and there are 50 samples for each class totalling 150 samples. Here is a sample taken from the data.

Understanding Entities

In the field of data science or machine learning and artificial intelligence, entities represent groups of data separated based on conceptual themes and/or data acquisition methods. An entity typically represents a table in a database, or a flat file, e.g.,comma separated variable (csv) file, or tab separated variable (tsv) file. Sometimes it is more efficient to represent the entities using a more structured formats like svmlight.[2] Each entity can contain multiple attributes. The raw data for each application can contain multiple such entities (Table 3.1).

In case of Iris data, we have only one such entity in the form of dimensions of sepals and petals of the flowers. However, if one is trying to solve this classification problem and finds that the data about sepals and petals alone is not sufficient, thenhe/she can add more information in the form of additional entities. For example more information about the flowers in the form of their colors, or smells or longevity of the trees that produce them, etc. can be added to improve the classification performance.

Understanding Attributes

Each attribute can be thought of as a column in the file or table. In case of Iris data, the attributes from the single given entity are *sepal length in cm*, *sepal width in cm*,

The structured formats like svmlight are more useful in case of sparse data, as they add significant overhead when the data is fully populated. A sparse data is data with high dimensionality (typically in hundreds or more), but with many samples missing values for multiple attributes. In such cases, if the data is given as a fully populated matrix,it

will take up a huge space in memory.

Sepal-length	**Sepal-width**	**Petal-length**	**Petal-width**	**Class label**
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3.0	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5.0	3.6	1.4	0.2	Iris-setosa
4.8	3.4	1.9	0.2	Iris-setosa
5.0	3.0	1.6	0.2	Iris-setosa
5.0	3.4	1.6	0.4	Iris-setosa
5.2	3.5	1.5	0.2	Iris-setosa
5.2	3.4	1.4	0.2	Iris-setosa
7.0	3.2	4.7	1.4	Iris-versicolor
6.4	3.2	4.5	1.5	Iris-versicolor
6.9	3.1	4.9	1.5	Iris-versicolor
5.5	2.3	4.0	1.3	Iris-versicolor
6.5	2.8	4.6	1.5	Iris-versicolor
6.7	3.1	4.7	1.5	Iris-versicolor
6.3	2.3	4.4	1.3	Iris-versicolor
5.6	3.0	4.1	1.3	Iris-versicolor
5.5	2.5	4.0	1.3	Iris-versicolor
5.5	2.6	4.4	1.2	Iris-versicolor
6.3	3.3	6.0	2.5	Iris-virginica

5.8	2.7	5.1	1.9	Iris-virginica
7.1	3.0	5.9	2.1	Iris-virginica
6.3	2.9	5.6	1.8	Iris-virginica
6.5	3.0	5.8	2.2	Iris-virginica
6.7	3.0	5.2	2.3	Iris-virginica
6.3	2.5	5.0	1.9	Iris-virginica
6.5	3.0	5.2	2.0	Iris-virginica
6.2	3.4	5.4	2.3	Iris-virginica
5.9	3.0	5.1	1.8	Iris-virginica

Understanding Data Types

Attributes in each entity can be of various different types from the storage and processing perspective, e.g., string, integer valued, datetime, binary (“true”/“false”, or “1”/“0”), etc. Sometimes the attributes can originate from completely different domains like an image or a sound file, etc. Each type needs to be handled separately for generatinga feature vector that will be consumed by the machine learning algorithm. We will discussthe details of this processing in Chap. 15. As discussed before, we can also come across sparse data, in which case, some attributes will have missing values. This missing data is typically replaced with special characters, which should not be confused with any of the real values. In order to process data with missing values, one can either fill those missing values with some default values, or use an algorithm that can work with missing data.

In case of Iris data, all the attributes are real valued numerical and there is no missing data. However, if we add additional entities like color, it would have enumerative type string features like *green*, *orange*, etc.

Representation and Visualization of the Data

Even after we have understood the data with the three hierarchical levels, we still do not know how the data is distributed and how it related to the output or class label. This marks the last step in preprocessing the data. We live in 3-dimensional world,so any

data that is up to 3 dimensional, we can plot it and visualize it.

However, when there are more than 3-dimensions, it gets tricky. The Iris data, for example, also has 4 dimensions. There is no way we can plot the full information in each sample in a single plot that we can visualize. There are couple of options in such cases:

1. Draw multiple plots taking 2 or three dimensions at a time.
2. Reduce the dimensionality of the data and plot upto 3 dimensions

Drawing multiple plots is easy, but it splits the information and it becomes harder to understand how different dimensions interact with each other. Reducing dimensionality is typically preferred method. Most common methods used to reduce the dimensionality are:

1. Principal Component Analysis or PCA
2. Linear Discriminant Analysis or LDA

1.11.3.1 Principal Component Analysis

We can only visualize the data in maximum of 2 or 3 dimensions. However, it is common practice to have the dimensionality of the data in tens or even hundreds.

In such cases, we can employ the algorithms just fine, as the mathematics on which they are based scales perfectly fine for higher dimensions. However, if we want to actually have a look at the data to see the trends in the distribution or to see the classification or regression in action, it becomes impossible. One way to do this is to plot the data in pairs of dimensions. However, in such case, we only see partial information in any one plot and it is not always intuitive to understand the overall trends by looking at multiple separate plots. In many situations, the real dimensionality of data is much less than what the dimensionality in which the data is presented. For example, check the 3-dimensional data plotted in Fig. 3.5 and the same data plotted with different perspective in Fig. 3.6. As can be seen in the second perspective, the data is actually only2 dimensional, if we can adjust the coordinates suitably. In other words, one can imagineall the data to be plotted on a singlepiece paper, which is only 2-dimensional and then holding it in skewed manner in a 3-dimensional space.

If we can find exact coordinates (X^r, Y^r) of the paper's orientation as linear combination of X, Y, and Z coordinates of the 3-dimensional space, we can reduce the dimensionality of the data from 3 to 2. The above example is for illustrations only and in most real situations the lower dimensionality is not reflected in such obvious manner. Typically the data does have some information in all the dimensions, but the extent of the

information in some dimensions can be very small compared to the information present in the other dimensions. For visualization purposes and also in real application purposes,it is quite acceptable to lose the information in those dimensions, without sacrificing any noticeable performance.

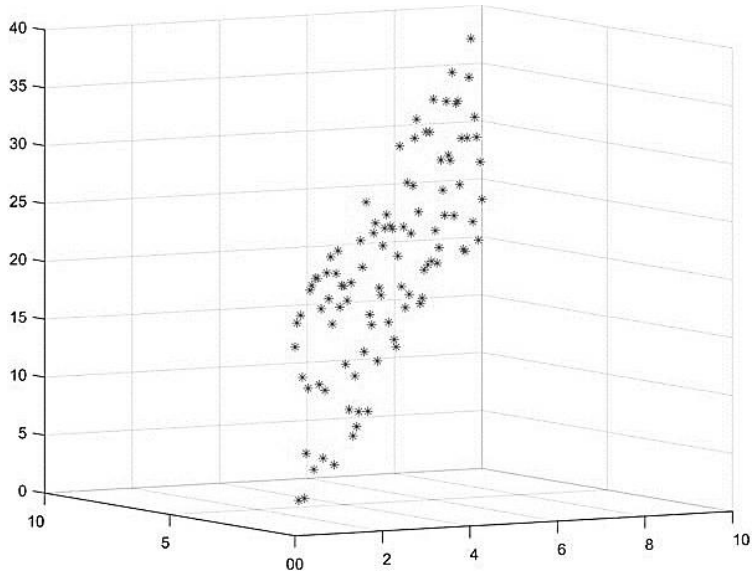


Fig. 3.5 dimensional data containing only 2-dimensional information with first perspective

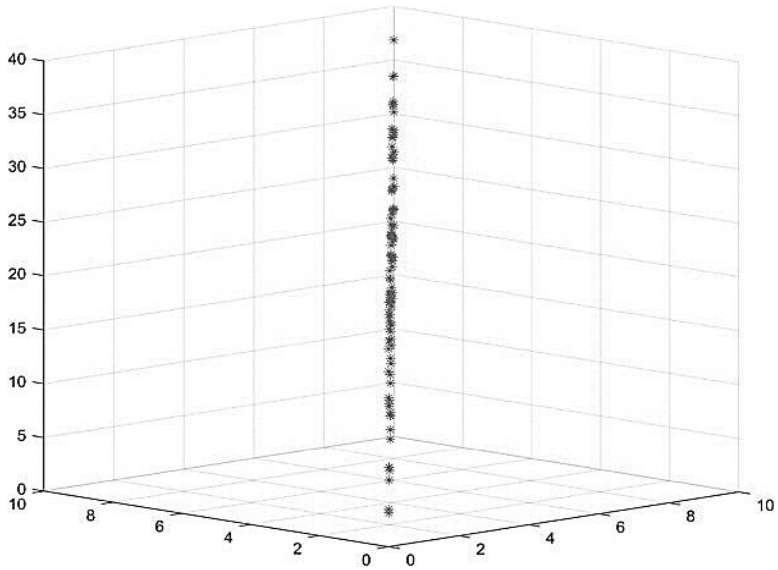


Fig. 3.6 3-dimensional data containing only 2-dimensional information with alternate perspective

As discussed in the previous chapter, the higher dimensionality increases the complexity of the problem exponentially, and such reduction in dimensionality forloss of some information is acceptable in most situations.

The mathematical process for finding the information spread across all the dimensions and then ranking the dimensions based on the information content is done using theory of principal component analysis or PCA. This theory is based on properties of matrices, specifically the process of *singular value decomposition(SVD)*. This process starts with first finding the dimension along which there is maximum spread or information content. If we have started with n-dimensions, then after first principal component is found the algorithm tries to find the next component of maximum spread in the remaining *n* 1 dimensional space that is orthogonal to the first component. The process continues till we reach the last dimension. The process also gives coefficient for each of the principal componentsthat represent the relative spread along that dimension. As all the components are found with deterministic and monotonically decreasing coefficient of spread, this representation is useful for any further analysis of the data.

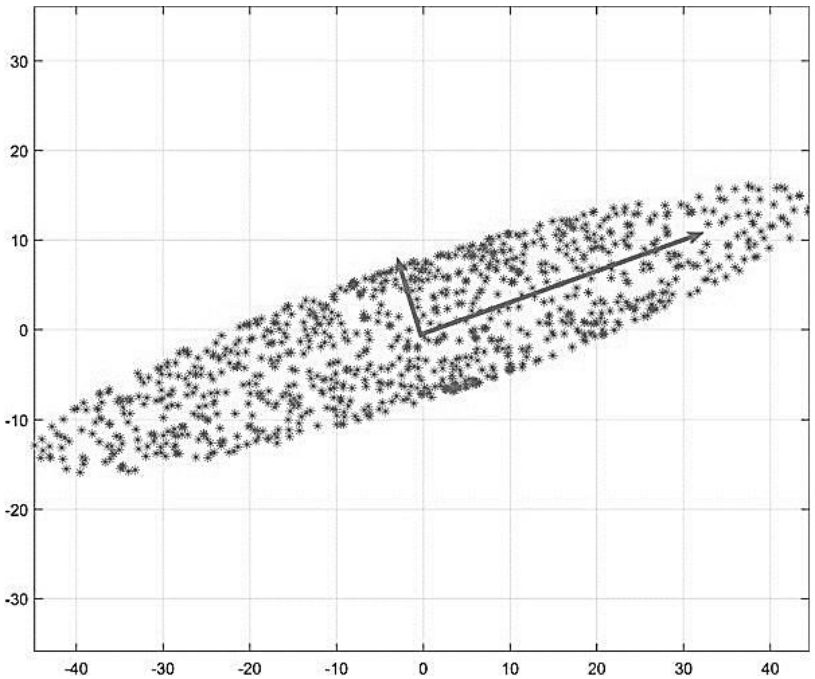


Fig. 3.7 2-dimensional data where PCA dimensionality is same but along different axes

Thus this process is not restricted to only reducing the dimensions, but to find the optimal dimensions that represent the data. As can be seen in Fig. 3.7 (principal

components are shown in red arrows), the original data and principal components are both 2-dimensional, but the representation of the data along principal components is different and preferable compared to original coordinates.

LINEAR DISCRIMINANT ANALYSIS

The way PCA tries to find the dimensions that maximize variance of the data, linear discriminant analysis or LDA tries to maximize the separation between the classes of data. Thus LDA can only effectively work when we are dealing with classification type of problem, and the data intrinsically contains multiple classes. Conceptually LDA tries to find the hyperplane in c 1 dimensions to achieve: maximize the separation of means of the two classes and minimize the variance of each class. Here c is number of classes. Thus, while doing this classification it finds a representation of the data in c 1 dimensions. For full mathematical details of theory one can refer to [5].

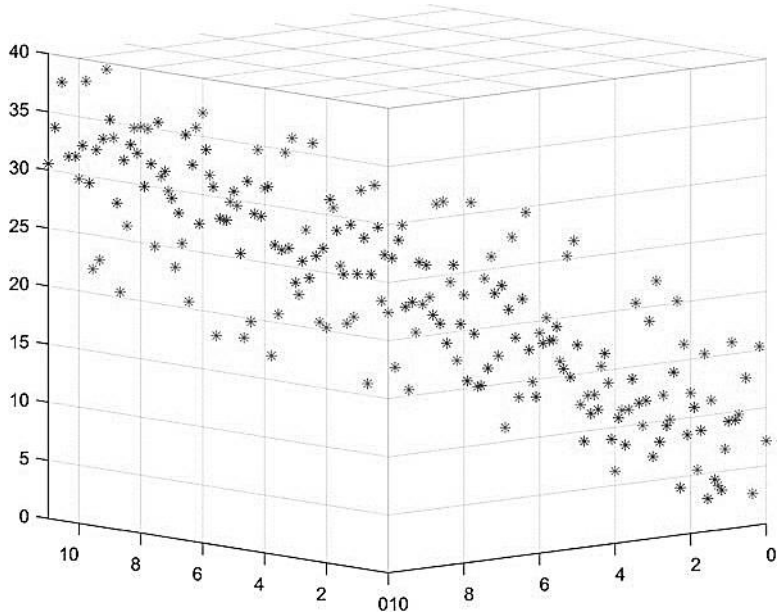
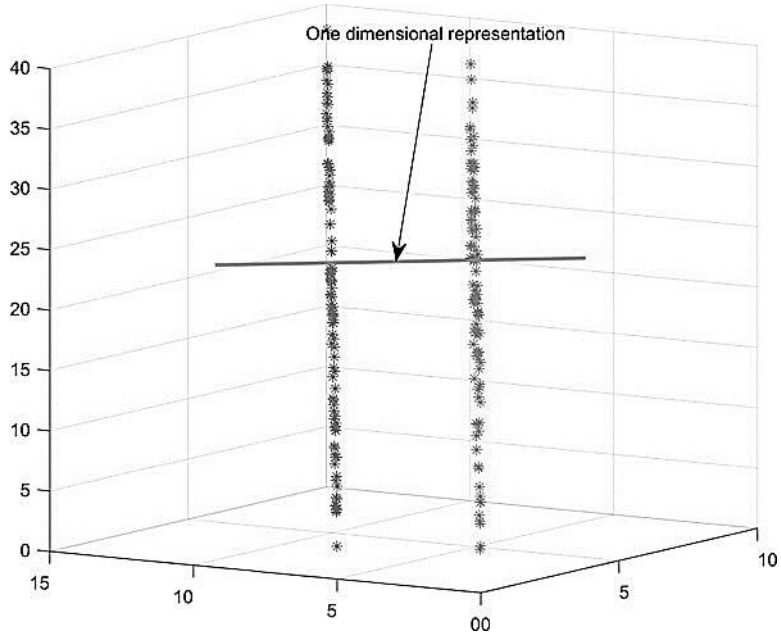


Fig. 3.8 3-dimensional data with 2 classes

Figure 3.8 shows 3 dimensional data in one perspective, where the classes are not quite separable. Figure 3.9 shows another perspective of the data where classes are separable. LDA finds precisely this perspective as linear combination of the features and creates a 1-dimensional representation of the data on a straight line where the classes are best separated. As there are only two classes, the LDA representation is 1-dimensional. The LDA representation is independent of the original dimensionality of the data.

As can be seen, sometimes this representation is bit extreme and not quite useful in understanding the structure of the data and then PCA would be a preferred choice.



UNIT II

MACHINE LEARNING METHODS

Linear methods - Regression - Classification - Perceptron and Neural networks – Decision trees - Support vector machines - Probabilistic models - Unsupervised learning - Featurization.

LINEAR METHODS

Introduction

In general machine learning algorithms are divided into two types:

1. Supervised learning algorithms
2. Unsupervised learning algorithms

LINEAR AND GENERALIZED LINEAR MODELS

The models that operate on strictly linear data are called linear models, and the models that use some nonlinear transformation to map original nonlinear data to linear data and then process it are called as generalized linear models. The conceptof linearity in case of supervised learning implies that the relationship between theinput and output can be described using linear equations. For unsupervised learning, the concept of linearity implies that the distributions that we can impose on the given data are defined using linear equations.

LINEAR REGRESSION

Linear regression is a classic example of strictly linear models. It is also called as polynomial fitting and is one of the simplest linear methods in machine learning. Let us consider a problem of linear regression where training data contains p samples. Input is n-dimensional as ($\mathbf{x}_i, i = 1, \ldots, p$) and $\mathbf{x}_i \in R^n$. Output is single dimensional as ($y_i, i = 1, \ldots, p$) and $y_i \in R$.

2.31.1 Defining the Problem

The method of linear regression defines the following relationship between input $\mathbf{x}_i$ and predicted output $\hat{y}_i$ in the form of linear equation as:

$$\hat{y} = \sum_{j=1}^{n} x_{ij} . w_j + w_0 \qquad \ldots (2.1)$$

2.3.2 Solving the Problem

Most commonly used method to find the weight parameters is to minimize the mean square error between the predicted values and actual values. It is called as least squares method. When the error is distributed as Gaussian, this method yields an estimate called as maximum likelihood estimate or MLE. This is the best unbiasedestimate one can find given the training data. The optimization problem can be defined as

$$\min \| y_i - \hat{y}_i \|^2 \qquad \cdots (2.3)$$

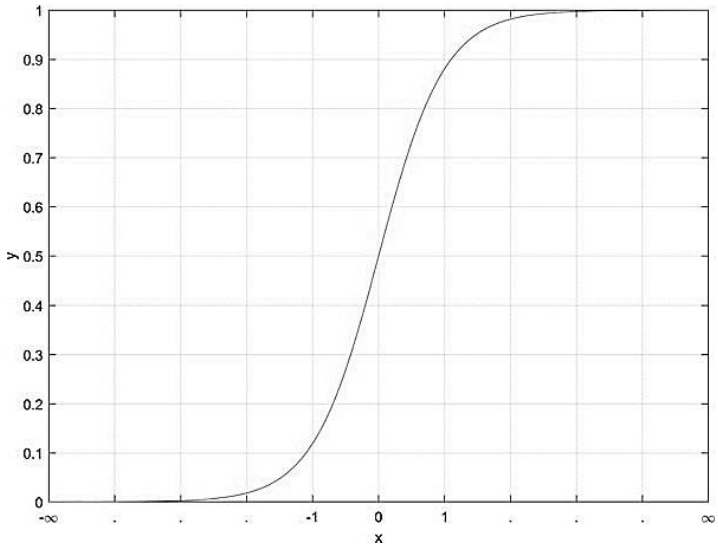


Fig. 2.1 Plot of logistic sigmoid function

REGULARIZED LINEAR REGRESSION

Regularization is a formal mathematical trickery that modifies the problem statement with additional constraints. The main idea behind the concept of regularization is to simplify the solution. The theory of regularization is typically attributed to Russian mathematician *Andrey Tikhonov*. In many cases the problems are what is referred to as *ill posed*. Ridge Regression..

GENERALIZED LINEAR MODELS (GLM)

The *Generalized Linear Models* or *GLMs* represent generalization of linear models by expanding their scope to handle nonlinear data that can be converted into linearform using suitable transformations.

Logistic Regression

The building block of logistic regression is the logistic sigmoid function *σ(x)* and is defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}} \qquad \text{... (2.7)}$$

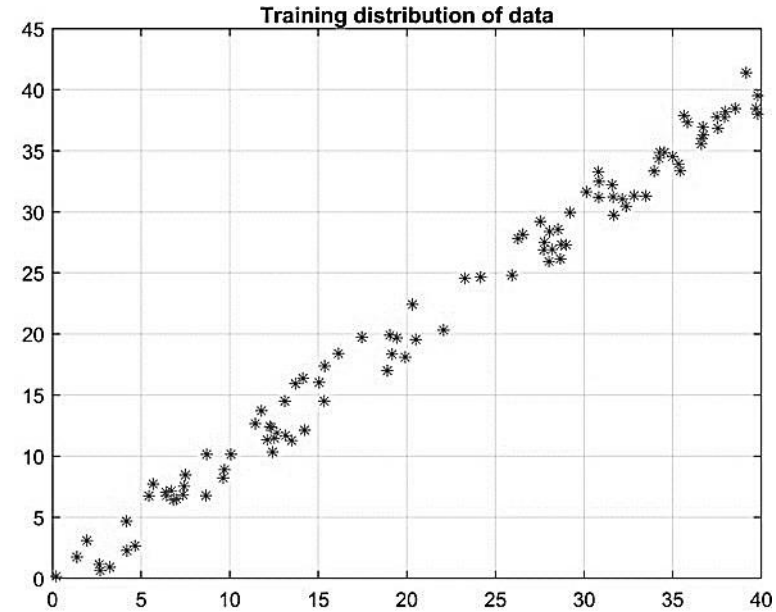
As the output is constrained between 0, 1 , it can be treated as a probabilistic measure.

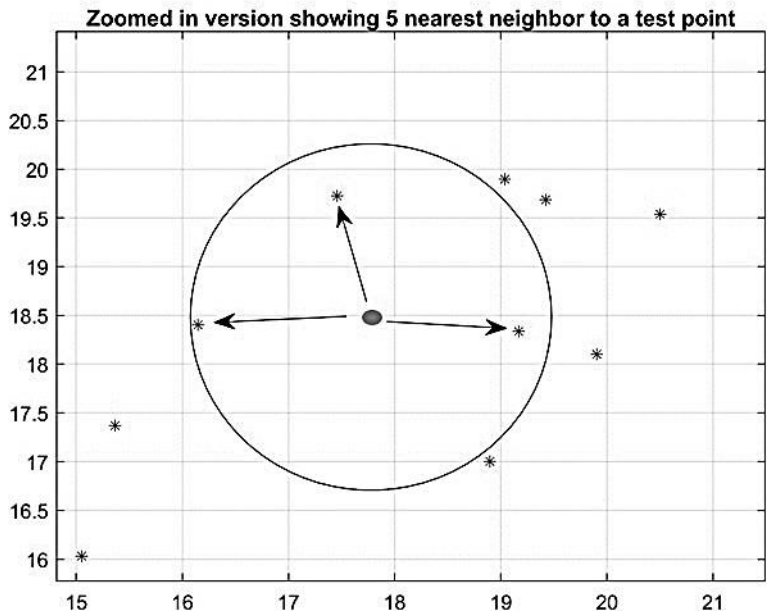
k-NEAREST NEIGHBOR (KNN) ALGORITHM

The KNN algorithm is not exactly an example of a linear method, but it is one of the simplest algorithms in the field of machine learning, and is apt to discuss it here in the first chapter of this part. KNN is also a generic method that can be used as classifier or regressor. Definition of KNN

This simplicity does come at a cost though. This lazy execution of the algorithm requires heavy memory footprint along with high computation to find the nearest

neighbors for each test point. However, when the data is fairly densely populated, and computation requirements can be handled by the hardware, KNN produces good results inspite of being expressed with overly simplistic logic.


Training distribution of data
45
40
35
30
25
20
15
10
5
0
0
5
10
15
20
25
30
35
40


Zoomed in version showing 5 nearest neighbor to a test point
21
20.5
20
19.5
19
18.5
18
17.5
17
16.5
16
15
16
17
18
19
20
21

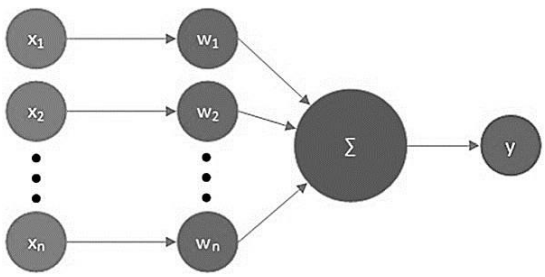
PERCEPTRON AND NEURAL NETWORKS

Perceptron was introduced by Rosenblatt [44] as a generalized computationframe-work for solving linear problems. It was quite effective, one of a kind machine at thetime and seemingly had unlimited potential.

Perceptron

Geometrically a single layered perceptron with linear mapping represents a linear plane in n-dimensions. In n-dimensional space the input vector is represented as $(x_1, x_2, \ldots, x_n)$ or **x**. The coefficients or weights in n-dimensions are represented as $(w_1, w_2, \ldots, w_n)$ or **w**. The equation of perceptron in the n-dimensions is then written in vector form as

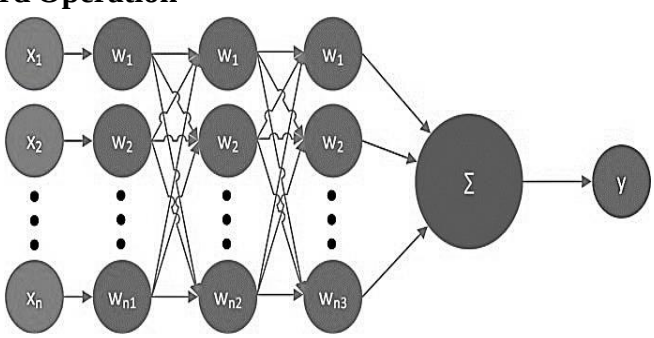
$$\mathbf{x}.\mathbf{w} = y \qquad \ldots(2.9)$$



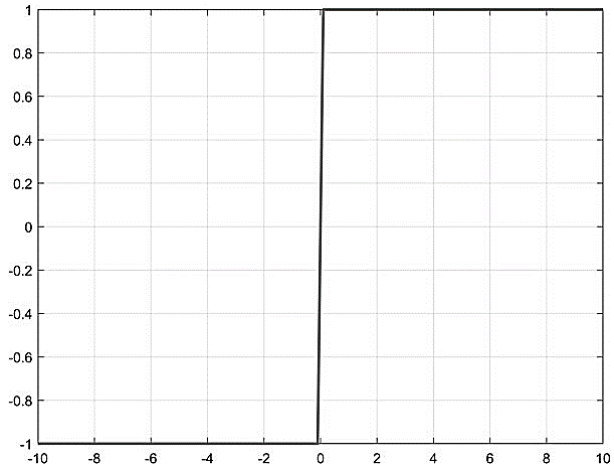
Multilayered Perceptron or Artificial Neural Network

Multilayered perceptron (*MLP*) seems like a logical extension of the single layer architecture, where we use multiple layers instead of single. Figure 2.4 shows an illustration of a generic MLP with *m* layers. Let *n*1 be the number of nodes in layer 1, which is same as the input dimensionality.

Feed forward Operation



1. Present the input to the neural network.
2. All the weights of the network are assigned some default value.



3. The input is transformed into output by passing through each node or neuron in each layer.

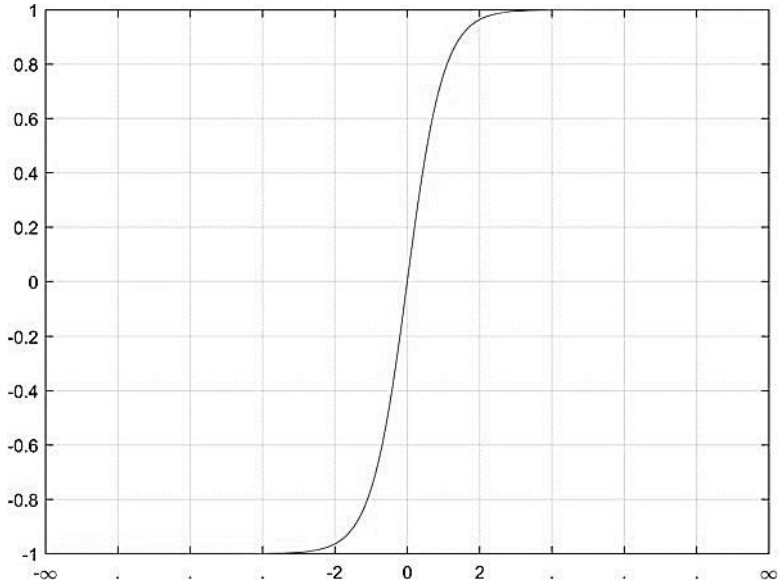


Fig. 2.7 Activation function *tanh*

4. The output generated by the network is then compared with the expected output or label.
5. The error between the prediction and label is then used to update the weights of each node.
6. The error is then propagated in backwards direction through every layer, to update the weights in each layer such that they minimize the error.

Thus backpropagation algorithm for training and feedforward operation for prediction mark the two phases in the life of neural network. Backpropagation-based training needs to be done in two different methods.

1. Online or stochastic method
2. Batch method

Online or Stochastic Learning

In this method a single sample is sent as input to the network and based on the output error the weights are updated. The optimization method most commonly used to update the weights is called stochastic gradient descent or *SGD* method. The use of stochastic here implies that the samples are drawn randomly from the wholedata set, rather than using them sequentially. The process can converge to desired accuracy level even before all the samples are used. It is important to understand that in stochastic learning process, single sample is used in each iteration and the learning path is more noisy. In some cases, rather than using a single sample, a mini-batch of samples is used. SGD is beneficial when the expected learning path can contain multiple local minima.

Batch Learning

In batch method the total data set is divided into a number of batches. Entire batch of samples is sent to the network before computing the error and updating the weights. After entire batch is processed, the weights are updated. Each batch process is called as one iteration.

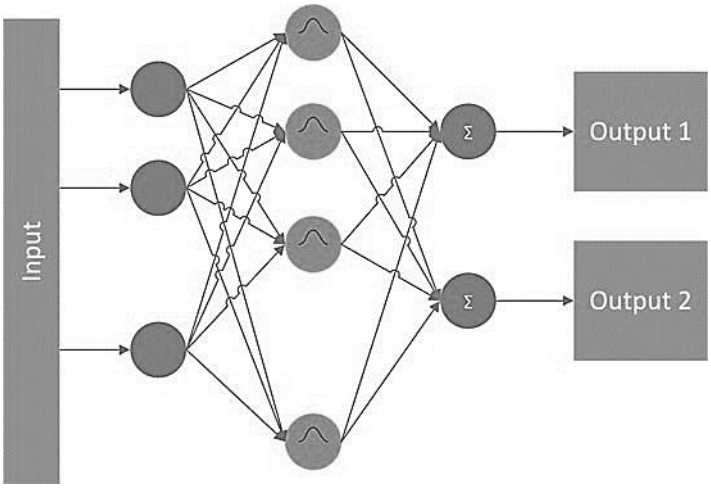
Hidden Layers

The concept of hidden layers needs a little more explanation. As such they are not directly connected with inputs and outputs and there is no theory around how many such layers are optimal in given application. Each layer in MLP transforms the input to a new dimensional space. Radial Basis Function Networks

Radial basis function networks *RBFN* or radial basis function neural networks *RBFNN* are a variation of the feedforward neural networks (we will call them as RBF networks to avoid confusion).

$$fRBF\ (x) = e^{-\beta\|x-\mu\|^2)} \qquad \ldots(2.10)$$

Above equation is defined for a scalar input, but without lack of generality it can be extended for multivariate inputs. μ is called as center and β represents the spread or variance of the radial basis function. It lies in the input space. Figure 2.9 shows theplot of the basis function. This plot is similar to Gaussian distribution.



Interpretation of RBF Networks

Aside from the mathematical definition, RBF networks have a very interesting interpretability that regular MLP does not have. Consider that the desired values of output form *n* number of clusters for the corresponding clusters in the input space.

OVERFITTING AND REGULARIZATION

1. Increasing number of hidden layers
2. Increasing the nodes in hidden layers
3. Using complex activation functions
4. Increasing the training epochs

L1 and L2 Regularization

Regularization approaches the problem using Lagrangian multiplier, where on top of minimizing the prediction error, we add another term in the optimization problem thatrestricts the complexity of the network with Lagrangian weighing factor λ.

Equations 2.11 and 2.12 show the updated cost function *C(x)* use of L1 and L2 type of regularizations to reduce the overfitting.

$$x) + \lambda \Sigma \|\mathbf{W}\|^2 \quad \ldots(2.12)$$

L(x) is the loss function that is dependent on the error in prediction, while W stand for the vector of weights in the neural network. The L1 norm tries to minimizethe sum of absolute values of the weights while the L2 norm tries to minimize the sum of squared values of the weights.

DROPOUT REGULARIZATION

This is an interesting method and is only applicable to the case of neural networks,while the L1 and L2 regularization can be applied to any algorithm. In dropout regularization, the neural network is considered as an ensemble of neurons in sequence, and instead of using fully populated neural network, some neurons are randomly droppedfrom the path. The effect of each dropout on overall accuracy is considered, and after some iterations optimal set of neurons are selected in the final models. As this technique actually makes the model simpler rather than adding more complexity like L1 and L2 regularization techniques, this method is quite popular, specifically in case of more complex and deep neural networks that we will study in later chapters.

DECISION TREES

Decision trees represent conceptually and mathematically a different approach towards machine learning. The other approaches deal with the data that is strictly numerical and can increase and/or decrease monotonically. The equations that define these approaches cannot process a data that is not numerical, e.g., categorical or string type. However, the theory of decision trees does not rely on the data being numerical.

Why Decision Trees?

Before going into the details of decision tree theory, let's understand why decision trees are so important. Here are the advantages of using decision tree algorithms for reference:

1. More human-like behavior.
2. Can work directly on non-numeric data, e.g., categorical.
3. Can work directly with missing data. As a result data cleaning step can be skipped.
4. Trained decision tree has high interpretability compared to abstract nature of trained

models using other algorithms like neural networks, or SVM, etc.

5. Decision tree algorithms scale easily from linear data to nonlinear data withoutany change in core logic.

6. Decision trees can be used as non-parametric model, thus hyperparameter tuningbecomes unnecessary.

Algorithms for Building Decision Trees

Most commonly used algorithms for building decision trees are:

- CART or Classification and Regression Tree
- ID3 or Iterative Dichotomiser
- CHAID or Chi-Squared Automatic Interaction Detector

CART or classification and regression tree is a generic term used for describingthe process of building decision trees as described by Breiman–Friedman [39]. ID3 is a variation of CART methodology with slightly different use of optimization method. CHAID uses a significantly different procedure and we will study it separately.

The development of classification trees is slightly different but follows similar arguments as a regression tree. Let's consider a two-dimensional space defined by axes (x_1, x_2). The space is divided into 5 regions (R_1, R_2, R_3, R_4, R_5) as showninfigure, using a set of rules as defined in Fig. 2.10.

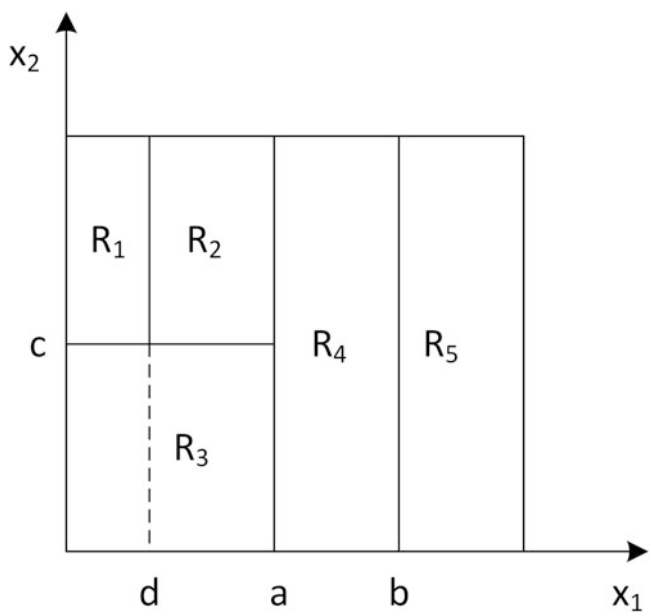


Fig. 2.10 Rectangular regions created by decision tree

Regression Tree

Regression trees are the trees that are designed to predict the value of a functionat given coordinates. Let us consider a set of N-dimensional input data $\{\mathbf{x_i}, i$ $1, \ldots, p$ and $\mathbf{x_i}$ $^n\}$. The corresponding outputs are

$\{y_i, i \quad 1, \ldots, p$ and $y_i\}$. It is required that in case of regression trees the input and output datais numerical and not categorical. Once given this training data, it is the job of thealgorithm to build the set of rules. How many rules should be used, what dimensions to use, when to terminate the tree are all the parameters that the algorithm needs tooptimize based on the desired error rate. ... (2.14)

Classification Tree

- Misclassification error
- Gini index
- Cross-entropy or deviance

Let there be "k" classes and "n" nodes. Let the frequency of class (m) predictions at each node (i) be denoted as f_{mi}. The fraction of the classes predicted as m at node$i$ be denoted as p_{mi}. Let the majority class at node m be c_m. Hence the fraction ofclasses c_m at node m would be $p_{mc}m$.

Decision Metrics

Let's define the metrics used for making the decision at each node. Differences in the metric definition separate the different decision tree algorithms.

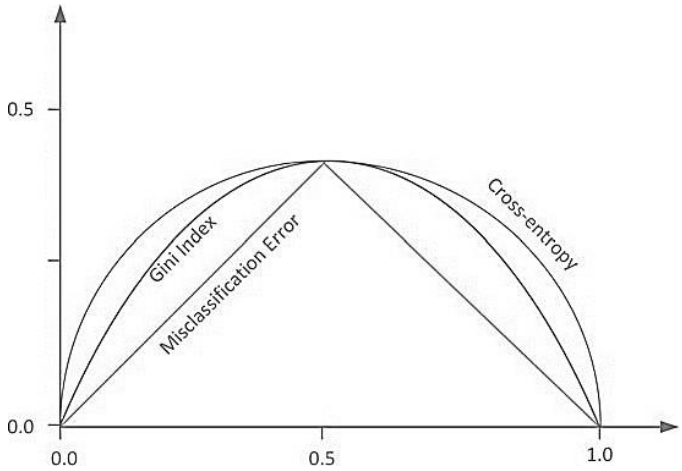
Misclassification Error

Based on the variables defined above the misclassification rate is defined as $1 - p_{mc}m$. As can be seen from the figure this rate is not a continuous function and hence cannot be differentiated. However, this is one of the most intuitive formulations and hence is fairly popular.

Gini Index

Gini index is the measure of choice in CART. Concept of the Gini index can be summarized as the probability of misclassification of a randomly selected input sample if

it was labelled based on the distribution



As the plot in Fig. 2.12 shows, this is a smooth function of the proportion and is continuously differentiable and can be safely used in optimization.

Cross-Entropy or Deviance

Cross-entropy is an information theoretic metric defined as

$$\varepsilon = -\sum_{m=1}^{m=k} p_{mi} \log(p_{mi}) \qquad \text{... (2.17)}$$

CHAID

Chi-square automatic interaction detector or *CHAID* is a decision tree technique that derives its origin in statistical chi-square test for goodness of fit. It was first published

by G. V. Kass in 1980, but some parts of the technique were already in use in 1950s. This test uses the chi-square distribution to compare a sample with a population and predict at desired statistical significance whether the sample belongs to the population. CHAID technique uses this theory to build a decision tree. Due to the use of chi-square technique in building decision tree, this method is quite different compared to any other types of decision trees discussed so far. Followingsubsection discusses the details of the algorithm briefly.

CHAID ALGORITHM

Training Decision Tree

We are not going into full mathematical details of building a decision tree using CART or ID3, but the following steps will explain the methodology in sufficient details and clarity.

Steps

1. Start with the training data.
2. Choose the metric of choice (Gini index or cross-entropy).
3. Choose the root node, such that it splits the data with optimal values of metrics into two branches.
4. Split the data into two parts by applying the decision rule of root node.
5. Repeat the steps 3 and 4 for each branch.
6. Continue the splitting process till leaf nodes are reached in all the branches with predefined stop rule.

Ensemble Decision Trees

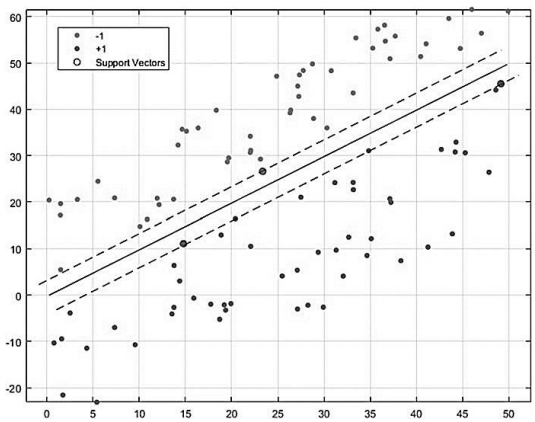
There are three main types of ensembles:

1. Bagging
2. Random forest
3. Boosting

SUPPORT VECTOR MACHINES

Theory of support vector machines or *SVMs* is typically attributed to Vladimir Vapnik. He was working in the field of optimal pattern recognition using statistical methods in the early 1960s. His paper with Lerner [52] on generalized portrait algorithm marks the beginning of the support vector machines. This method was primarily designed to solve the problem of binary classification using constructionof optimal hyperplane that separates the two classes with maximum separation.

Support Vector Machines



Extension to Multi-Class Classification

As per the conceptual setup of SVM, it is not directly extensible for solving the problem of multi-class classification. However there are few approaches commonly used to extend the framework for such case.

Theory of SVM

Let us consider a binary classification problem with n-dimensional training data set consisting of p pairs $(\mathbf{x}_i, y_i)$, such that $\mathbf{x}_i \in R^n$ and $y_i \in \{-1,+1\}$. Let theequation of the hyperplane that separates the two classes with maximal separation be given as

$$(\mathbf{w} \cdot \mathbf{x}) - w_0 = 0 \qquad \ldots(2.18)$$

Here $\mathbf{w} \in R^n$ same as $\mathbf{x}$. For the samples belonging to each class we can write

$$(w.x_i) - \omega_0 \begin{cases} > 1, & if\ y_i = 1 \\ \leq 1, & if\ y_i = -1 \end{cases} \qquad \ldots(2.19)$$

The two equations can be combined into a single equation as

$$y_i[(\mathbf{w} \cdot \mathbf{x}) - w_0] \geq 1,\ \ i = 1,\ldots,p \qquad \ldots(2.20)$$

The above equation can be solved to get multiple solutions for weight vector.

In order to get the solution that also minimizes the weight vector we impose a constraint to minimize $\Phi(\mathrm{w})$. Where $\Phi(\mathrm{w})$ is given as

$$\Phi(\mathbf{w}) = \|\mathbf{w}'\|^2 \qquad \ldots(2.21)$$

where $\mathbf{w}'$ is an $(n+1)$ dimensional vector that is combination of $\mathbf{w}$ and w_0.

Thus we have arrived at a point where we can define the optimization problem that needs to be solved. To obtain the precise equation of the hyperplane, we need to minimize the functional $\Phi(\mathbf{w}^{\mathrm{r}})$ with constraints defined in Eq. 2.22. Typically such problems are solved using the technique of *Lagrangian*, that combines the functional that is to be minimized and the constraints into a single Lagrange functional that needs to be minimized,

PROBABILISTIC MODELS

The probabilistic models are mainly classified into two types:

1. Generative
2. Discriminative

The probabilistic approaches (discriminative as well as generative) are also sliced based on two universities of thought groups:

1. Maximum likelihood estimation
2. Bayesian approach

Discriminative Models

We will first discuss the distinction between these two classes from the perspective of discriminative models and then we will turn to generative models.

Maximum Likelihood Estimation

The maximum likelihood estimation or *MLE* approach deals with the problems at the face value and parameterizes the information into variables. The values of the variables that maximize the probability of the observed variables lead to the solution of the problem. Let us define the problem using formal notations. Let there be afunction $f(\mathbf{x}\theta)$ that produces the observed output y. $x \in \Re^n$ represent the input on which we don't haveany control over and $\theta \in \Theta$ represent a parameter vector that can be single or multidimensional. The MLE method defines a likelihood function denoted as $L(y|\theta)$. Typically the likelihood function is the joint probability of theparameters and observed variables as $L(y|\ \theta)\ P(y;\theta)$. The objective is to find the optimal values for θ that maximizes the likelihood function as given by

$$\theta^{MLE} = \arg\max_{\theta g \Theta}\{L(y|\theta)\} \qquad \text{... (2.29)}$$

or,

$$\theta^{MLE} = \arg\max_{\theta g \Theta}\{P(y;\theta\} \qquad \text{... (2.30)}$$

This is a purely frequentist approach that is only data dependent.

Bayesian Approach

Bayesian approach looks at the problem in a different manner. All the unknowns are modelled as random variables with known prior probability distributions. Let us

denote the condi|tional prior probability of observing the output y for parameter vector θ as $P(y\ \theta)$. The marginal probabilities of these variables are denoted as $P(y)$ and $P(\theta)$. The joint probability of the variables can be written in terms of conditional and marginal probabilities as

$$P(y;\theta) = P(y|\theta) \cdot P(\theta) \qquad \text{...(2.31)}$$

The same joint probability can also be given as

$$P(y;\theta) = P(\theta\,|y) \cdot P(y) \qquad \text{…(2.32)}$$

Here the probability $P(\theta/y)$ is called as posterior probability. Combining Eqs. 2.31 and 2.32

$$P(\theta\,|y) \cdot P(y) = P(y|\theta) \cdot P(\theta) \qquad \text{…(2.33)}$$

rearranging the terms we get

$$P(\theta|y) = \frac{P(y|\theta).\,P(\theta)}{P(y)} \qquad \text{... (2.34)}$$

$$(posterior) = \frac{(prior).(likelihood)}{(evidence)} \qquad \text{... (2.35)}$$

Gamma distribution is also one of the very highly studied distribution in theory of statistics. It forms a basic distribution for other commonly used distributions like chi-squared distribution, exponential distribution etc., which are special cases of gamma distribution. It is defined in terms of two parameters: α and β. The pdf of gamma distribution is given as

$$P_{gamma}(x|\alpha,\beta) = \frac{\beta^{\alpha}x^{\alpha-1}e^{-\beta x}}{\Gamma(\alpha)} \qquad \text{... (2.45)}$$

where $x > 0$ and $\alpha, \beta > 0$. The simple definition of $\Gamma(\alpha)$ for integer parameter is given as a factorial function as

$$\Gamma(n) = (n-1)! \qquad \text{…(2.46)}$$

The same definition is generalized for complex numbers with positive real parts as

$$\Gamma(\alpha) = \int_{0}^{\infty} x^{\alpha-1} e^{-\beta x} dx \qquad \text{... (2.47)}$$

This function also follows the recursive property of factorials as

$$\Gamma(\alpha) = (\alpha - 1)\Gamma(\alpha - 1) \qquad \text{…(2.48)}$$

Figure 2.19 shows plots of the pdf for variable α and β values.

The cdf of gamma function cannot be stated easily as a single valued function, but rather is given as sum of an infinite series as

$$C_{gamma}(x|\alpha, \beta) = e^{-\beta x} \sum_{1=\alpha}^{\infty} \frac{(\beta x)^i}{\mathrm{i}!} \qquad \text{... (2.49)}$$

Figure 2.20 shows plots of the cdf for variable α and β values similar to the ones shown for pdf.

UNSUPERVISED LEARNING

In this chapter, we will learn about the following aspects of unsupervised learning:

1. Clustering
2. Component Analysis
3. Self Organizing Maps (SOM)
4. Autoencoding neural networks

Clustering

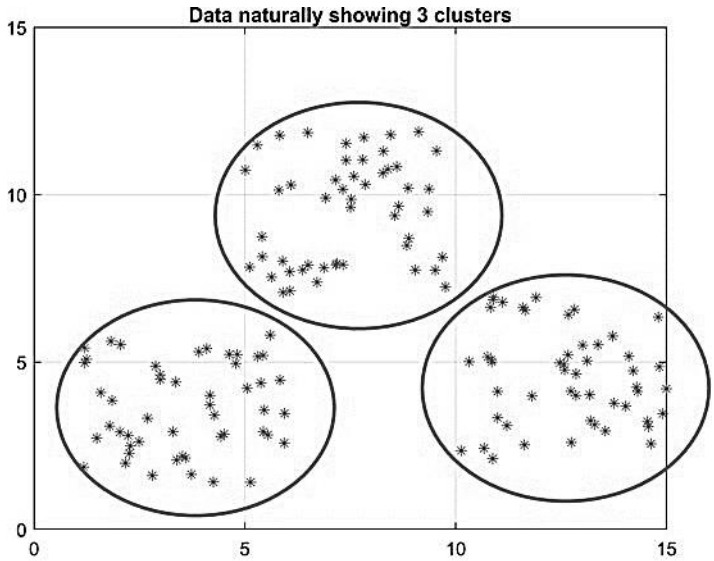
Clustering is essentially aggregating the samples in the form of groups. The criteria used for deciding the membership to a group is determined by using some form of metric or distance.k-Means Clustering

The *k*-means clustering algorithm can be summarized as follows:

1. Start with a default value of *k*, which is the number of clusters to find in the given data.
2. Randomly initialize the *k* cluster centers as *k* samples in training data, such that there are no duplicates.
3. Assign each of the training samples to one of the *k* cluster centers based on a chosen distance metric.
4. Once the classes are created, update the centers of each class as mean of all the samples in that class.
5. Repeat steps 2–4 until there is no change in the cluster centers.

Optimizations for *k*-Means Clustering

1. Change the stopping criterion from absolute “no change” to cluster centers to allow for a small change in the clusters.
2. Restrict the number of iterations to a maximum number of iterations.
3. Find the number of samples in each cluster and if the number of samples is less than a certain threshold, delete that cluster and repeat the process.



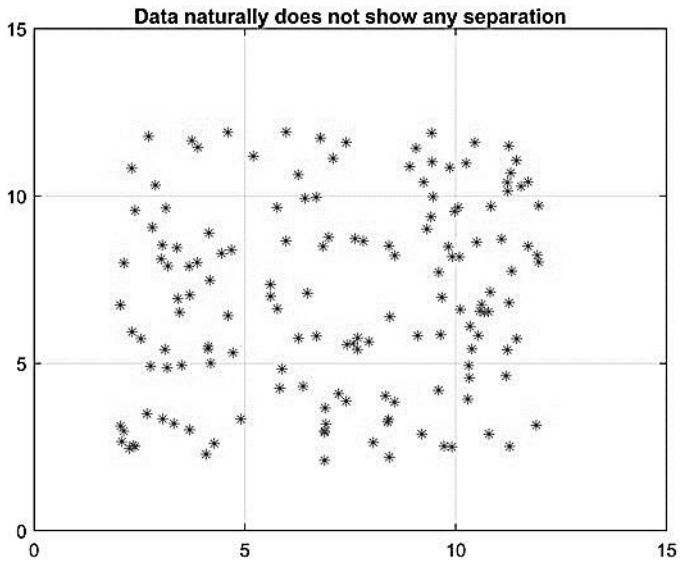


Fig. 2.22 Figures showing the extreme cases of data distribution for clustering

4. Find the intra-cluster distance versus inter-cluster distance, and if two clusters are too close to each other relative to other clusters, merge them and repeat the process.

5. If some clusters are getting too big, apply a threshold of maximum number of samples in a cluster and split the cluster into two or more clusters and repeat the process.

FEATURIZATION

UCI: Adult Salary Predictor

UCI Machine Learning Repositoty is one of the well-known resources for finding sample problems in machine learning. It hosts multiple data sets targeting variety of different problems.

Feature Details

1. *age*: continuous.
2. *workclass*: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
3. *fnlwgt*: continuous.
4. *education*: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th–8th, 12th, Masters, 1st–4th, 10th, Doctorate, 5th–6th, Preschool.
5. *education-num*: continuous.
6. *marital-status*: Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse.
7. *occupation*: Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.
8. *relationship*: Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
9. *race*: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
10. *sex*: Female, Male.
11. *capital-gain*: continuous.
12. *capital-loss*: continuous.
13. *hours-per-week*: continuous.
14. *native-country*: United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador,

Trinadad and Tobago, Peru, Hong, Holand-Netherlands.

Most of the feature names are descriptive of their meaning, but for the rest, the reader is advised to check [6].

Expected outcome is binary classification as,

1. salary > $50K
2. salary <= $50K

UNIT III

MACHINE LEARNING IN PRACTICE

RANKING

Introduction

Ranking at heart is essentially sorting of information. Unconsciously we are always ranking things around us based on some metrics.

3.2.1 Measuring Ranking Performance

The algorithm that can rank the items in strictly non-increasing order of relevance.

Let there be n number of items that need to be ranked and each has a relevance score of r_i, i = 1, 2,...,n. A simple measure, called as cumulative gain (CG) is defined on these relevances as,

$$CG = \sum_{i=1}^{n} r_i$$

CG essentially represents the overall quality of aggregate search results. As there is no positional factor in the expression, CG does not provide any information about theranking. Hence a modified measure is defined as discounted cumulative gain or DCG.DCG is defined as,

$$DCG = \sum_{i=1}^{n} \frac{r_i}{\log_b i + 1}$$

The base b of the logarithm is typically used as 2. The reason for using i +1 instead of i is to account for the case when i = 1. In this case, the denominator would be 0 and the metric would be meaningless.

Item No.	Relevance Score	Discounted relevance score
1	0.4	0.4
2	0.25	0.16
3	0.1	0.05
4	0.0	0.0
5	0.3	0.12

In this case they are ranked arbitrarily

Table 3.2 Sample set of items ideally ranked with non-increasing relevance score

Item No.	Relevance Score	Discounted relevance score
1	0.6	0.6
2	0.56	0.35
3	0.4	0.2
4	0.3	0.13
5	0.25	0.1
6	0.22	0.08
7	0.13	0.04
8	0.1	0.032
9	0.0	0.0
10	0.0	0.0

If we sum the discounted scores, then we get DCG as 1.20. As we can see this ranking is not ideal, as the highest ranked item is at rank 7 and all the items are fairly arbitrarily ranked. Now, let’s fix the rankings and recalculated the discounted relevance scores as shown in Table 3.2.

Now, if we sum the discounted relevance scores, we get 1.51, which is significantly higherthan the previous score, that was produced with random ordering of the items. Thus we have a good metric in the form of DCG that can compare two different

sets of rankings if we have corresponding relevance scores.

There is one more problem with DCG and that is the actual value of a score is quite arbitrary based on the range of values of relevance scores. For example, if recalibrate our relevance scores to range from 0 to 5 instead of 0–1 as before, our ideal DCG would jump from 1.51 to 7.66 and our random DCG would jump from 1.20 to 6.01. All these values are fairly arbitrary and the value of 6.01 has not much meaning without knowing the ideal value of DCG, which is 7.66.

Ranking Search Results and Google's Pagerank

All the metrics discussed above hold true and provide a good measure of ranking quality, only if we have a single numeric measure of the relevance score for each item. However, coming up with such score is not always a trivial task. Rather the goodness of such measure is the true deciding factor in building a good ranking system.

Techniques Used in Ranking Systems

Information retrieval and text mining are the core concepts that are important in the ranking system that relate to searching websites or movies, etc. We will look specifically at a technique called as **keyword identification/extraction** and word cloud generation. This method underpins most of the text mining systems and knowledge of these techniques is invaluable.

Keyword Identification / Extraction

Figure 3.1 shows a word cloud from the famous Gettysburg address by Abraham Lincoln in 1861. A word cloud is a graphical representation of the keywords identified from a document or set of documents. The size of each keyword represents the relative importance of it.

Let's look at the steps required to find these importances

Steps in Building a Word Cloud

1. Cleaning up the data. This step involves removing all the formatting characters as well as punctuation marks.
2. Normalization of the data. This step involves making all the characters lower case, unless it is required to distinguish the upper case letters from lower case. Another aspect of normalization is lemmatization or stemming. This step involves applying grammar to find the root word for every word form. For example root word for running is run and so on.

3. Removal of stop words. This step involves removing all the common words that are typically used in any text and have no meaning of their own. For example a, them for, to, and etc.
4. Compute the frequency of occurrence of each of the remaining words.
5. Sort the words in descending order of frequency.
6. Plot the top-n keywords graphically to generate the word cloud.

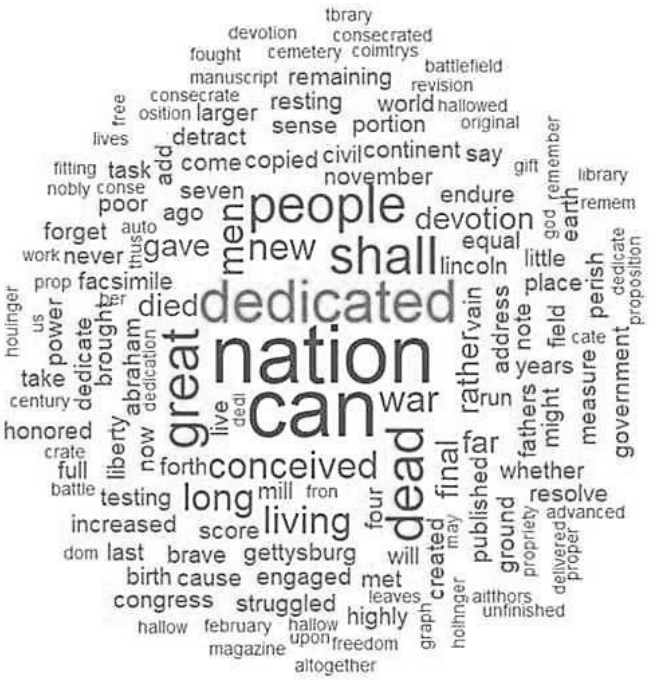


Fig. 3.1 Word cloud from famous Gettysburg address by Abraham Lincoln

Above analysis works well for a single document case. When there are multiple documents, then there is one more technique that is useful. It is called as TF-IDF, or term frequency - inverse document frequency. Term frequency is calculated in similar way as described above. The inverse document frequency is an interesting concept and it is based on the notion of how much important the given keyword is in the current document as opposed to other documents. Thus a word that appears frequently across all the documents will have lowvalue of IDF in any of these documents. However, when a keyword appears frequently inonly one document, but is almost not present in the rest of the documents, then its importance in the given document is even higher. Mathematically, both the terms are defined as,

$$\text{tf} = \frac{\text{frequency of the world in given document}}{\text{Maximum frequency of any word in the given document}}$$

$$\text{idf} = \frac{\text{frequency of the world in given document}}{\text{Total number of documents in which the word appears}}$$

RECOMMENDATIONS SYSTEMS

Introduction

The core mathematical theory underlying modern recommendation systems is commonly called as collaborative filtering. Companies like Amazon and Netflix have influenced this genre of machine learning significantly for personalizing shopping and movie watchingexperiences respectively.

Collaborative Filtering

A mathematical process of predicting the interests or preferences of a given user based on a database of interests or preferences of other users. Very loosely speaking, these predictions are based on the nearest neighbor principle. Users with similar interests in one aspect tend to share similar interest in other similar aspects.

Collaborative filters always deal with 2-dimensional data in the form of a matrix. One dimension being the list of users and the other dimension being the entity that is being liked or watched or purchased, etc Table 3.3 shows a representative table. The table is always partially filled, representing the training data. The goal is to predict all the missing values.

Solution Approaches

The core information that can be possibly available is of three types.

Information Types

1. Information about the users in the form of their profiles. The profiles can have key aspects like age, gender, location, employment type, number of kids, etc.
2. Information about the interests. For movies, it can be in the form of languages, genres, lead actors and actresses, release dates, etc.
3. Joint information of users' liking or rating their interests as shownin the Table 3.1.

Algorithm Types

1. Algorithms exploiting the topological or neighborhood information. These algorithms are primarily based on the joint historical information about the users'

ratings. They use nearest neighbor type methods to predict the unknown ratings. These algorithms cannot work if the historical data is missing.

2. Algorithms that exploit the structure of the relationships. These methods assume a structural relationship between users and their ratings on the interests. These relationships are modelled using probabilistic networks or latent variable methods like component analysis (principal or independent) or singular value decomposition. These methods use the joint information as well as the separate information about the user profiles and interest details. These methods are better suited to tackle sparse data and ill conditioned problems, but they miss out neighborhood information.

3. Hybrid approach. Most recommendation systems go through different phases of operation,where in the beginning the ratings data is not available and the only data that is available is user profiles and interest details. This is typically called as cold start. The neighborhood based algorithms simply cannot operate in such situations. Hence these hybrid systems start with algorithms that are more influenced by the structural models in the early stages and later on combine the advantages of neighborhood based models.

Amazon's Personal Shopping Experience

In order to understand the problem from Amazon's perspective, we need to explain the problem in more detail.

Context Based Recommendation

Aspects Guiding the Context Based Recommendations

1. Suggesting other similar products that are cheaper than the product selected. So, if the cost is the only aspect stopping the user from buying the selected item, the recommended item can solve the problem.

2. Suggesting a similar product from a more popular brand. This might attract user to buya potentially more expensive product that is coming from more popular brand, so more reliable or better quality.

3. Suggesting a product that has better customer reviews.

4. Suggesting a set of products that are typically bundled with the selected product. Thesesuggestions would right away increase the potential sale. For example suggesting carry bag or battery charger or a memory card when selected item is a digital camera.

DESIGNING AND TUNING MODEL PIPELINES

After Featurization to chose a Technique or Algorithm to use for the desired application.

Then the available data is split into two or three sets

- Training ◊ used for training the model
- Validation ◊ optional validation set is used to tune the parameters,
- Testing ◊ is used to predict the performance metrics of the algorithm

Designing and Tuning Model Pipelines involves following steps

- Choosing the Technique or Algorithm
- Choosing Technique for Adult Salary Classification
- Splitting the Data
- Stratified Sampling
- Training
- Tuning the Hyperparameters
- Accuracy Measurement
- Explain ability of Features
- Practical Considerations
- Data Leakage
- Coincidence and Causality
- Unknown Categories

3.3.1 Choosing the Technique or Algorithm

Choosing Technique for Adult Salary Classification

- As the application dictates we have to narrow our scope to classification algorithms. Regression algorithms can always be used in classification applications with addition of threshold, but are less preferred.
- Decision tree based methods are typically better suited for problems that have categorical features as these algorithms inherently use the categorical information. Most other algorithms like logistic regression, support vector machines, neural networks, or probabilistic approaches are better suited for numerical features.
- Tt is better to start with a **simpler algorithm** to begin with to establish a baseline performance.
- Single decision tree can be used as one of the simplest possible starting algorithm; however, random forest decision tree is also a good choice.

Splitting the Data

- Once an algorithm is selected for use, the next step is to separate the whole labelled training data into 2 or 3 sets, called as training, (optional) validation, and test.
- Typically the division of the three sets in percentage is done as: 60–20–20, or 70–15–15.

Stratified Sampling

The splitting is always done in random fashion and not sequential manner to have statistically even spread of data between all the three sets. Stratified sampling ensures a certain known distribution of classes in split parts. When we are dealing with say n number of classes, the number of samples per class is not always same.

When the original distribution is unbalanced, there are two choices:

1. Ignore a number of samples from the classes that have more samples to matchthe classes that have less number of samples.
2. Use the samples from the classes that have less number of samples repeatedlyto match with number of samples from the classes that have larger number of samples.

Example: **Training Set**

There are 24,720 ◊ samples from the class <=50K

7841 samples ◊ from the class >50K contains total of 32,561 samples

Test Tes-> 12,435 samples from the class <=50K

3846samples from the class >50K with total of 16,281 samples.

Training

Once the data is split into training and test sets, the training set is used to train the algorithm and build the model. Each algorithm has its own specific method associated withit for training.

The parameters of the model are also classified into two main types:

1. The parameters that can be computed using the training process to minimize the error in the prediction.
2. The parameter that cannot be directly computed using the training process. These parameters are also called as **hyperparameters**. Each unique set of hyperparameters can be thought of as a different model in itself.

Tuning the Hyperparameters

- The hyperparameters are typically a set of parameters that can be unbounded, theoretically choose any number between 1 and ∞.
- These bounds are created based on multiple constraints like computation requirements, dimensionality, and size of data, etc

Explain ability of Features

Most of the machine learnt models, specifically the neural net type models, lack this reasoning and interpretability completely. One must accept the outcome as it stands basedon the accuracy metrics that are provided based on the test data. However, this lack of reasoning or explain ability and interpretability came into strong criticism when the machine learnt models started to replace the older heuristic models.

Practical Considerations

Either the data is static, or it is in the form of time series. Either data is strictly linear or it can be converted into linear with suitable link function or it is purely nonlinear. The number of classes defined in the training data cannot be different from the ones in

test data.

Data Leakage

- If the data is static then there is a set of static algorithms that one can choose from. Examples of static problem could be classification of images into ones that contain cars and ones that don't.

For Example Consider a business of auto mechanic. Here are some of the columns in the data:

1. Number of visits
2. Year of manufacture of the car
3. Make of the car
4. Model of the car
5. Miles on the odometer
6. Amount of sale
7. Method of payment
8. Category of customer

Coincidence and Causality

There are some probabilistic methods proposed based on conditional dependence between features, but all such methods make some assumptions on the dependency and causality.

We can circumvent this situation by taking following measures:

1. Using cross validation even if hyperparameter tuning is not used. This splits the data in multiple different combinations thereby reducing the chance of coincidental features getting more importance.
2. Using ensemble methods as opposed to single model methods to improve the robustness and reduce coincidence.
3. Applying feature interpretation as an additional step after the model training to make sure all the important features have domain specific explain ability

Unknown Categories

Existence of missing data is a common occurrence in case of training classification problems. sometimes, it may happen that a new category is encountered only in the test

set, which wascompletely missing in the training or validation set. Most models can fail in such situations.However, if the model is built to take into consideration of this possibility this failure can beavoided. Here are few examples of handling such situation:

PERFORMANCE MEASUREMENT

To evaluate the performance or quality of the model, different metrics are used, and thesemetrics are known as performance metrics or evaluation metrics.

- **Performance Metrics for Regression / Numerical error**
  - Mean Absolute Error
  - Mean Squared Error
  - Normalized Error
- **Performance Metrics for Classification / Categorical error**
  - Accuracy
  - Precision and Recall
    - F-Score
    - Confusion Matrix
  - Receiver Operating Characteristics (ROC) Curve Analysis
- **Steps in Hypothesis Testing**
  - A/B Testing

Performance Metrics for Regression / Numerical error

These are the simplest form of metrics. When we have a list of expected values, say (yi,i = 1,...,n) and we have a list of predicted values, say (ˆy,i = 1,...,n). The error in prediction can be given using following metrics:

Mean Absolut e Error

Mean absolute error is defined as:

$$e_{mae} = \frac{\sum_{i=1}^{i=n} |y - y_i|}{n}$$

- Use of only Mean Error is typically avoided, as it can lead to unusually low values due to cancellation of the negative and positive errors.
- MAE is much more robust for the outliers. One of the limitations of MAE is that it is not differentiable, so for this, we need to apply different optimizers such as Gradient Descent. However, to overcome this limitation, another metric can be used, which is Mean Squared Error or MSE.

Mean Squared Error

MSE typically penalizes less number larger errors (outliers) more heavily compared to more number of smaller errors. One can choose to use either of them based on specific problem or use both.

Root Mean Squared Error

Root mean squared error is defined as:

$$e_{mae} = \sqrt{\frac{\sum_{i=1}^{i=n} \lceil y_i - y_l \rfloor}{n}}$$

RMSE reduces the sensitivity of the error to few outliers, but still is more sensitive compared to the MAE.

Normalized Error

- In many cases, all the above error metrics can produce some arbitrary number between -∞ and ∞.
- Typically the bounds used are (-1to+1), (0–1), or (0–100). This way, even a single instance of normalized error can make sense on its own. All the above error definitions can have their own normalized counterpart.

It can be formulated as t can be formulated as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total number of preductions}}$$

To quantitatively define the metrics, let's define certain parameters. Let n_{ij} be the number of samples originally of class i that are classified as class j. With joint perspective, the metrics are given in terms of accuracy A as,

$$A = \frac{n_{11} + n_{22}}{n_{11} + n_{12} + n_{21} + n_{22}}$$

Precision and Recall

We need to define few more terms for defining the metrics from perspective of any one class.

- Let TP be the number of true positives from the perspective of class 1. Hence, TP =n_{11}Let FP be the false positives from the perspective of class 1.
- False positive means the sample is actually from class 2, but is classified as class 1.
- Hence FP = n_{21}. They are also called as false calls.
- Let TN be the number of false negatives from the perspective of class 1. True negative means the sample is actually from class 2 and is classified as class 2. Hence TN = n_{22}.
- Let FN be the number of false negatives from the perspective of class 1. False negative means the sample is actually from class 1, but is classified as class 2. They are also alternatively called as misses. Hence FN = n_{12}.

Now we can define the two metrics from perspective of class 1 as precision, P and recall, R.

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

$$\mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

Rules of Thumb for Understanding Precision and Recall

1. Precision can be interpreted as how many samples actually belong to class 1 from the pool of samples that are classified as class 1.
2. Recall can be interpreted as how many samples actually belong to class 1 from the pool of samples that actually belong to class 1.

F-Score

In order to combine these two metrics into a single metric, sometimes another metric is defined as F-score, which is essentially harmonic mean of precision and recall. It is sometimes called as F-measure or F1 score, but they all mean the same quantity.

Confusion Matrix

$$F = \frac{P.R}{P + R}$$

		Actual	
		1	0
Predicted	1	True Positives (TP)	False Positives (FP)
	0	False Negative (FN)	True Negatives (TN)

A confusion matrix is a tabular representation of prediction outcomes of any binary classifier, which is used to describe the performance of the classification model on

a set of test data when true values are known. The confusion matrix is simple to implement, but the terminologies used in this matrix might be confusing for beginners. A typical confusion matrix for a binary classifier looks like the below image

Hypothesis Testing

- Hypothesis testing is essentially a probabilistic process of determining whether a given hypothesis is true.

Steps in Hypothesis Testing

1. First step in the process involves defining what is called as null hypothesis H0. Null hypothesis is essentially the default random behavior. null hypothesis would be: Example: "weather predictions by the local news channel are random and purely based on chance."

2. Second step is to gather experimental or observational data. In the case of example, thatwould mean the difference between the weather predictions recorded every day from the local news channel and actual observed weather conditions. These observations constitutea random variable, called as test statistic.

3. Third step involves computation of the probability (also called as P-value) that the test statistic actually proves the null hypothesis.

4. Fourth step involves comparing the P-value with the predetermined level of significance, denoted as a. (Typically value of a lies between 5 and 1%) to either accept or reject thenull hypothesis. The level of significance can be thought of as probability of test statistic lying outside of the expected region if null hypothesis is true.

A/B Testing

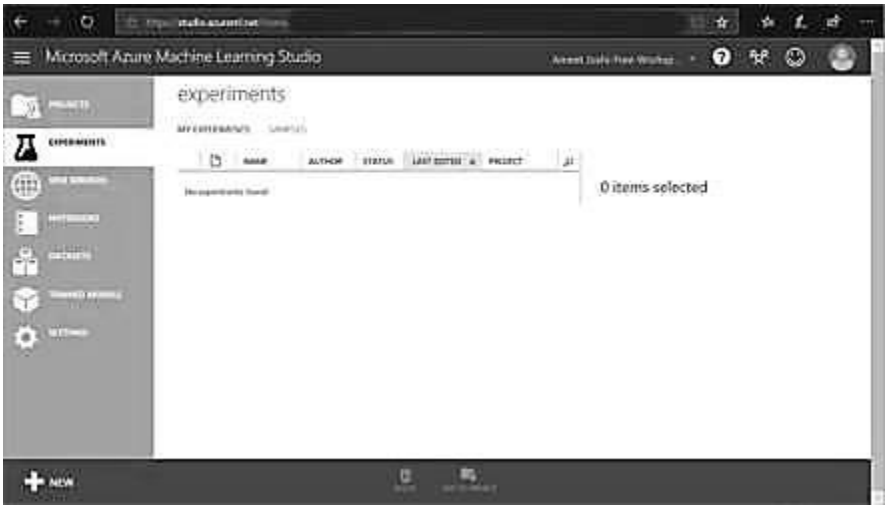
We can extend the concept of hypothesis testing to what is called as A/B testing. In A/B testing, rather than using a default null hypothesis, we use hypothesis A, which is the outcome from the model-A. Then compare this with the outcome of model-B as second hypothesis.

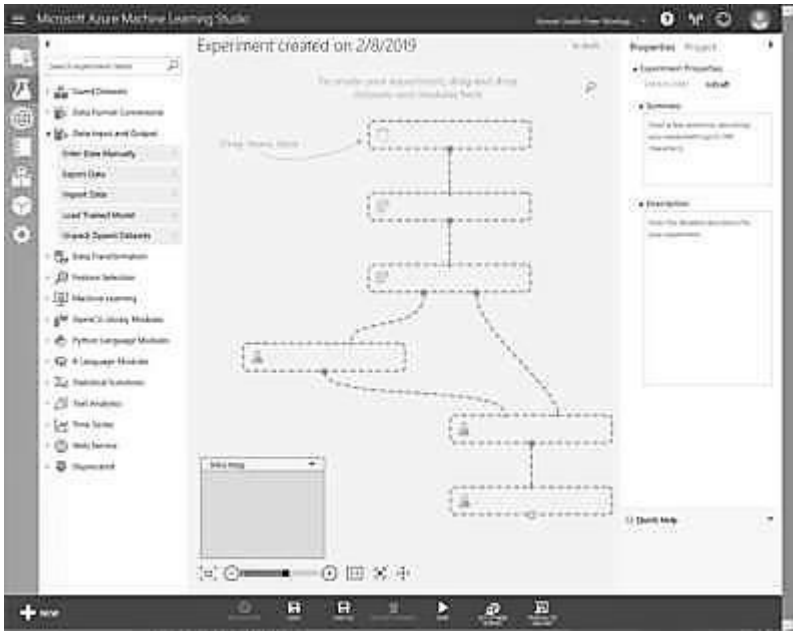
AZURE MACHINE LEARNING- AML

The first place one needs to visit is: https://studio.azureml.net/.

Fig. 3.4 Sign In screen for Azure machine learning studio

sign in with a Microsoft account,e.g., hotmail.com or outlook.com, etc. Once you sign in, you are presented with a screen as below





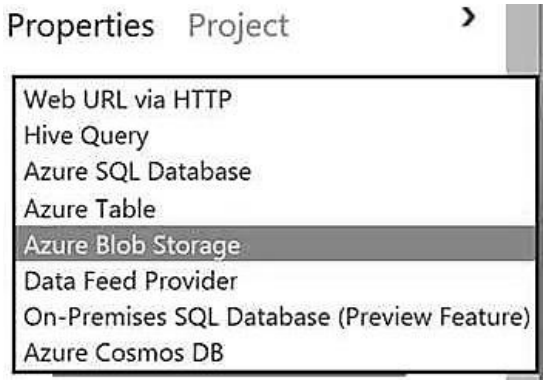
BUILDING ML PIPELINE USING AML STUDIO

This involves following steps

- Get the Data
- Data Preprocessing
- Training the Classifier Model

Get the Data

The first step in the process is to import the data. AML provides various different options to import data using Import Data block as shown below



Various data import options

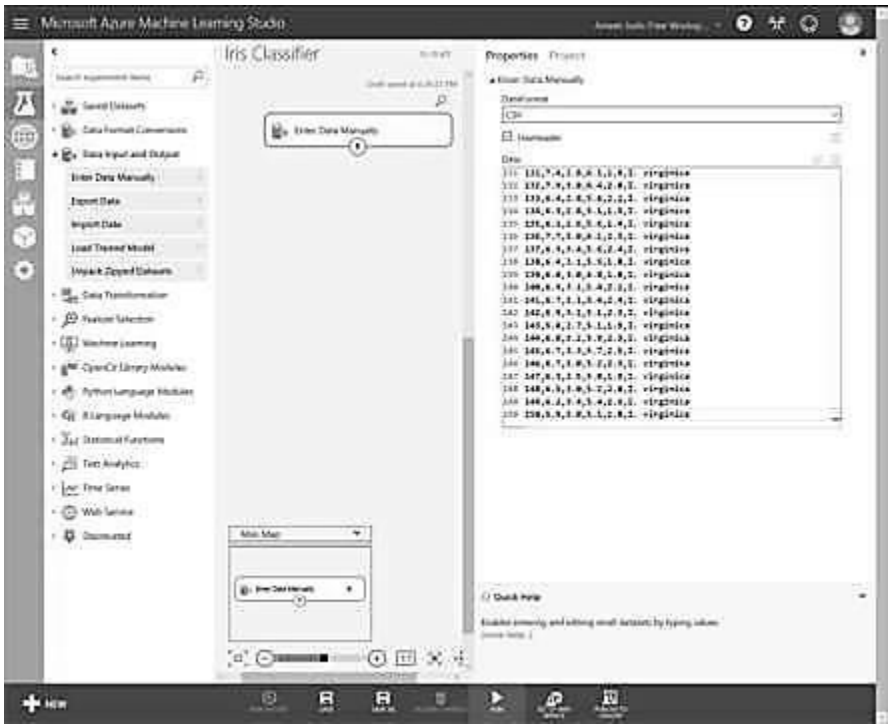


Fig: 3.5 Entering the data manually

Iris data is quite small, we will just use the block named Enter Data Manually. This block can be brought to the experiment canvas by drag and drop. Once you click on the block on the right panel it shows options on how you want to enter the data. Let's choose the default option of CSV and paste the CSV data in Data block as shown in Fig. 3.5

Data Preprocessing

1. Use Select Columns in Dataset block in Data Transformation->Manipulation in the left panel.
2. Use the block two times to select features and label separately.

To select the columns, just click on Launch column selector in right panel. That brings the UI shown in Fig. 3.6 Use the UI twice to select features in set and labels

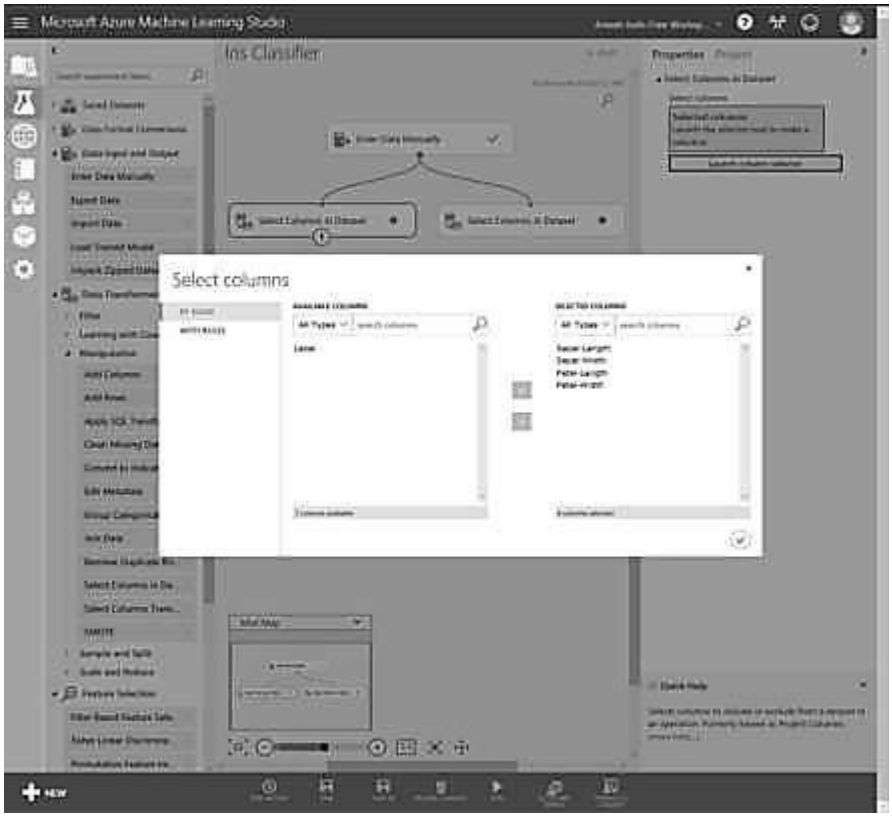


Fig. 3.6 UI for selecting columns

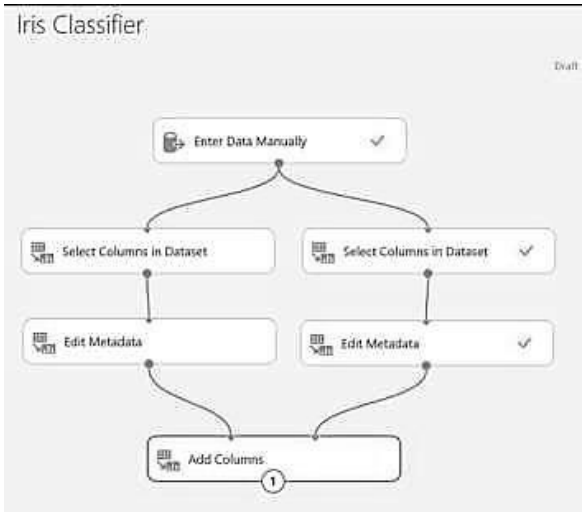


Fig. 3.7 ML pipeline at the end of data preprocessing
Fig. 3.8 Visualization of the labels

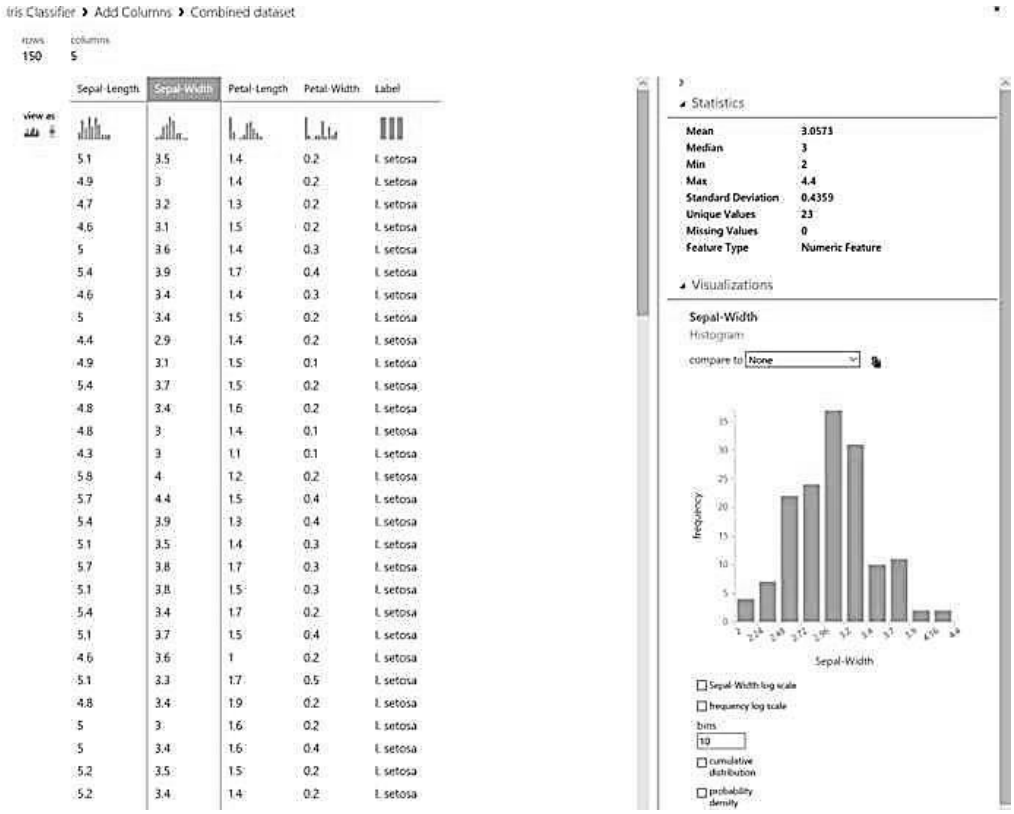


Fig. 3.9 Visualization of one of the features

Fig. 3.10 ML pipeline till training the model

3.5.2 Scoring and Performance Metrics

The next step in the processing is to score the test set with the trained model. We can use the block Score Model from Machine Learning -> Score on left panel.It takes the two inputs as trained model and test data. After we score the test set we cancompare the test results with the expected classes from the known labels.

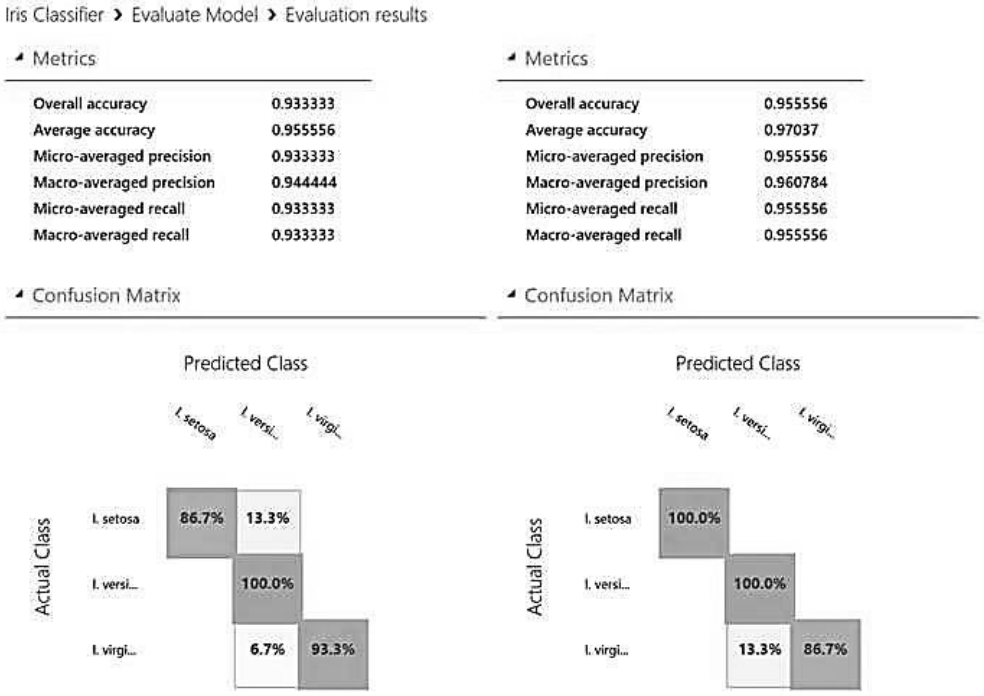


Fig. 3.14 Visualization of the evaluate block with comparative results from both the models

OPEN SOURCE MACHINE LEARNING LIBRARIES

Here is a list of some of the more popular machine learning libraries available in open source:

1. **Scikit-learn :** Scikit-learn, also called as sklearn is one of the most popular machine learning library and it also has support for most types of models. This is built on Python ecosystem.
2. **TensorFlow :**TensorFlow is a library published by Google as open source. It supports multiple languages including Python, C#, Java, etc. TensorFlow is more targeted towards deep learning applications and does not support quite a few of classical machine learning techniques.
3. **Theano :**Theano is another machine learning library that is targeted for deep learning applications. It is primarily based on Python ecosystem.
4. **CNTK Microsoft Cognitive Toolkit**, also called CNTK, is another deep learning

targeted machine learning library published as open source from Microsoft. CNTK supports Python and C++.

5. **Keras:** Keras is an interesting option. It provides an even higher level interface on top ofexisting libraries like Theano, TensorFlow, and CNTK. It provides unified commands to builddeep learning pipeline using either of three platforms using Python.
6. **Caffe:** Convolutional Architecture for Fast Feature Embedding or Caffe is another machine learning library targeted towards neural networks and deep learning written for Python.
7. **Torch:** Yet another deep learning library that is primarily written for another language called Lua. Lua has deeper ties with C, and hence Torch is also compatible with C or C++.

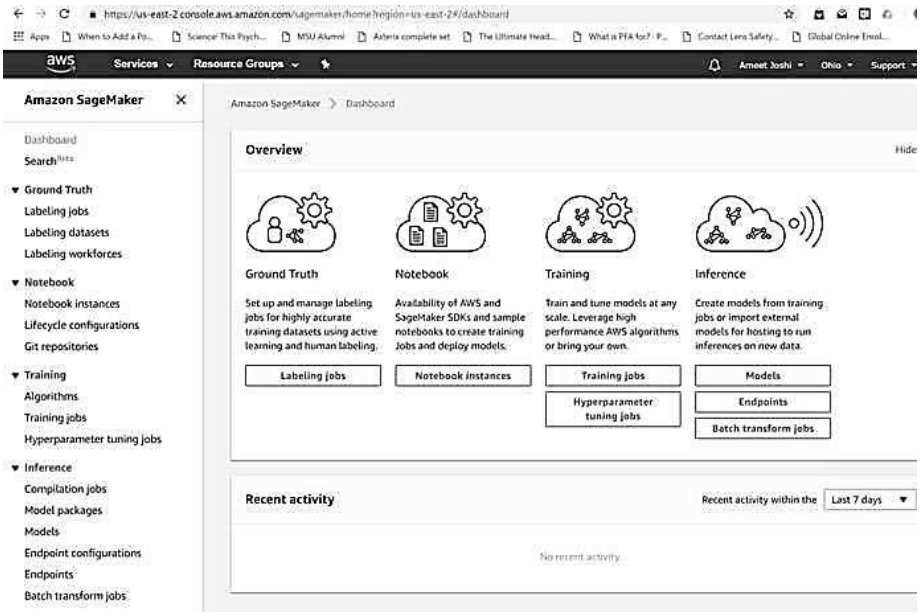
Scikit-Learn Library

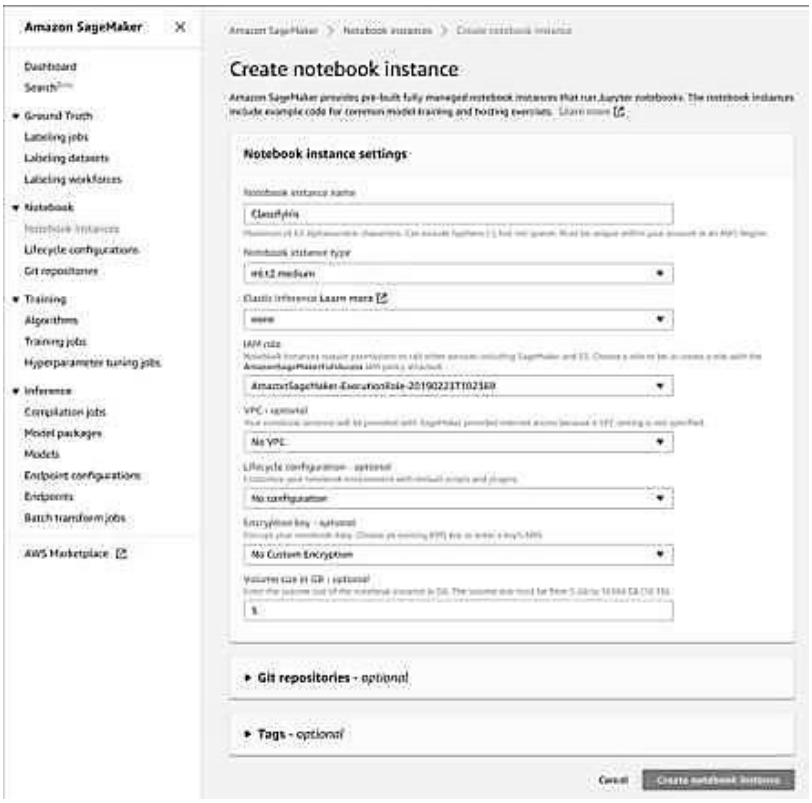
Scikit-learn or sklearn library is built on core Python and few additionallibraries like NumPy, SciPy, and Pandas. The plotting library Matplotlib is also needed. Here we are going to see

- Developing enviroinment
- Importing Data
- Data Preprocessing
- Splitting the Data Using Stratified Sampling
- Training a Multiclass Classification Model
- Computing Metrics
- Using Alternative Model

AMAZON'S MACHINE LEARNING TOOLKIT: SAGEMAKER

Setting Up Sagemaker


Amazon SageMaker
Dashboard
Search
Ground Truth
Labeling jobs
Labeling datasets
Labeling workforces
Notebook
Notebook instances
Lifecycle configurations
Git repositories
Training
Algorithms
Training jobs
Hyperparameter tuning jobs
Inference
Compilation jobs
Model packages
Models
Endpoint configurations
Endpoints
Batch transform jobs
Services
Resource Groups
Ameet Joshi
Ohio
Support
Amazon SageMaker > Dashboard
Overview
Hide
Ground Truth
Set up and manage labeling jobs for highly accurate training datasets using active learning and human labeling.
Labeling jobs
Notebook
Availability of AWS and SageMaker SDKs and sample notebooks to create training jobs and deploy models.
Notebook instances
Training
Train and tune models at any scale. Leverage high performance AWS algorithms or bring your own.
Training jobs
Hyperparameter tuning jobs
Inference
Create models from training jobs or import external models for hosting to run inferences on new data.
Models
Endpoints
Batch transform jobs
Recent activity
Recent activity within the
Last 7 days
No recent activity

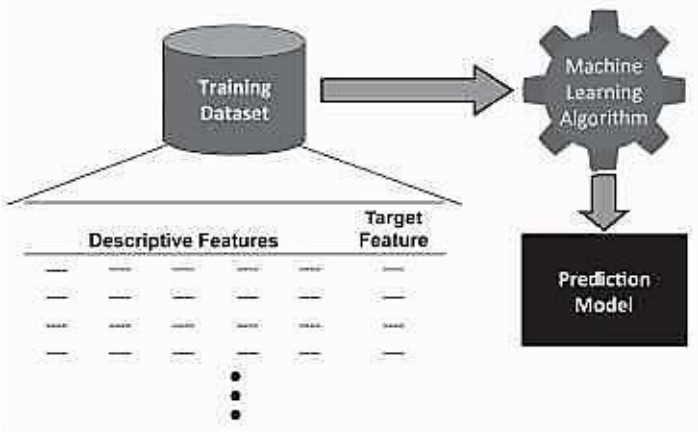

Amazon SageMaker
Dashboard
Search
Ground Truth
Labeling jobs
Labeling datasets
Labeling workforces
Notebook
Notebook instances
Lifecycle configurations
Git repositories
Training
Algorithms
Training jobs
Hyperparameter tuning jobs
Inference
Compilation jobs
Model packages
Models
Endpoint configurations
Endpoints
Batch transform jobs
AWS Marketplace
Create notebook instance
Notebook instance settings
Notebook instance name
Classifyiris
Notebook instance type
ml.t2.medium
Elastic Inference Learn more
none
IAM role
AmazonSageMaker-ExecutionRole-20190223T102169
VPC - optional
No VPC
Lifecycle configuration - optional
No configuration
Encryption key - optional
No Custom Encryption
Volume size in GB - optional
5
Git repositories - optional
Tags - optional
Cancel
Create notebook instance

UNIT IV
MACHINE LEARNING AND DATA ANALYTICS

Machine Learning for Predictive Data Analytics - Data to Insights to Decisions – Data Exploration - Information based Learning - Similarity based learning - Probability based learning - Error based learning - Evaluation - The art of Machine learning to Predictive Data Analytics..

MACHINE LEARNING FOR PREDICTIVE DATA ANALYTICS

Modern organizations collect massive amounts of data. For data to be ofvalue to an organization, they must be analyzed to extract insights that can be used to make better decisions. The progression from data to insights to decisions is illustrated in Figure 4.1. Extracting insights from data is the job of data analytics.



(a) Learning a model from a set of historical instances

(b) Using a model to make predictions

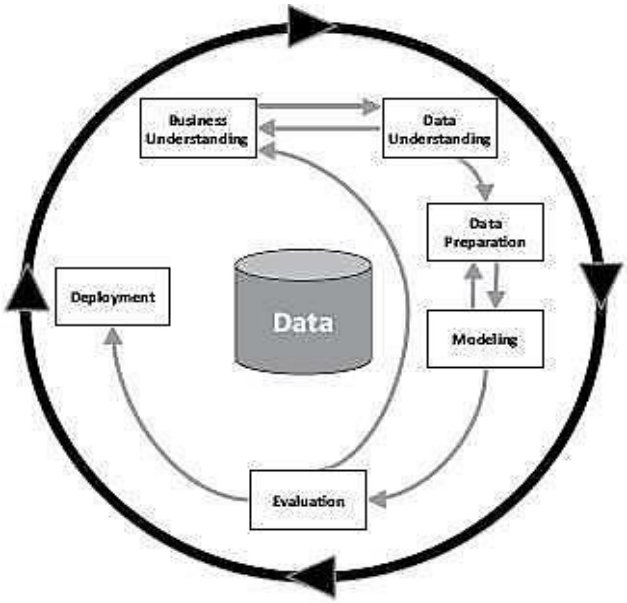
How Does Machine Learning Work?

- Machine learning algorithms work by searching through a set of possible prediction models for the model that best captures the relationship between the descriptive features and the target feature.
- An obvious criteria for driving this search is to look for Models that are consistent with the data.
- At least two reasons why just searching for consistent models is not sufficienorderder to learn useful prediction models.

Table 4.2 A simple retail dataset

ID	BBY	ALC	ORG	GRP
1	no	no	no	couple
2	yes	no	yes	family
3	yes	yes	no	family
4	no	no	yes	couple
5	no	yes	yes	single

The Predictive Data Analytics Project Lifecycle: Crisp-DM



Business Understanding:

Predictive data analytics projects never start out with thegoal of building a prediction model. Instead, they are focused on things like gainingnew customers, selling more products, or adding efficiencies to a process. So, duringthe first phase in any analytics project, the primary goal of the data analyst is to fullyunderstand the business (or organizational) problem that is being addressed, and then To design a data analytics solution for it.

Predictive Analytics Tools

Predictive Analytics Software Tools have advanced analytical capabilities like Text Analysis, Real-Time Analysis, Statistical Analysis, Data Mining, Machine Learning modeling and Optimization.

Libraries for Statistical Modeling and Analysis

- Scikit-learn
- Pandas
- Stats model
- NLTK (Natural Language Processing Tool Kit)
- GraphLab

- Neural Designer

Open-Source Analytical Tools

- SAP Business Objects
- IBM SPSS
- Halo Business Intelligence
- Daiku-DSS
- Weka

DATA TO INSIGHTS TO DECISIONS

This chapter we present an approach to developing analytics solutions that address specific **business problems and** the data structures that are required to build predictive analytics models, and in particular the **analytics base table (ABT).**

Converting a business problem into an analytics solution

Involves answering the following key questions:

1. What is the business problem?
2. What are the goals that the business wants to achieve?
3. How does the business currently work?
4. In what ways could a predictive analytics model help to address the business problem?

DATA EXPLORATION

The Data Quality Report

A data quality report includes tabular reports that describe the characteristics of each featurein an ABT using standard statistical measures of **central tendency and variation.**

The tabular reports are accompanied by data visualizations:

- A histogram for each continuous feature in an ABT
- A bar plot for each categorical feature in an ABT.

(a) Continuous Features

Feature	Count	% Miss.	Card.	Min.	1st Qrt.	Mean	Median	3rd Qrt.	Max.	Std. Dev.
___	___	___	___	___	___	___	___	___	___	___
___	___	___	___	___	___	___	___	___	___	___
___	___	___	___	___	___	___	___	___	___	___

(b) Categorical Features

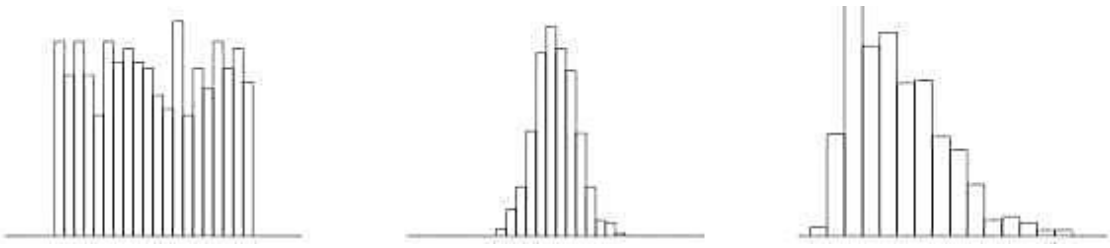
Feature	Count	% Miss.	Card.	Mode	Mode Freq.	Mode %	2nd Mode	2nd Mode Freq.	2nd Mode %
___	___	___	___	___	___	___	___	___	___
___	___	___	___	___	___	___	___	___	___
___	___	___	___	___	___	___	___	___	___

For categorical features, we should:

- Examine the mode, 2nd mode, mode %, and 2 nd mode % as these tell us the most common levels within these Features and will identify if any levels dominate the dataset.

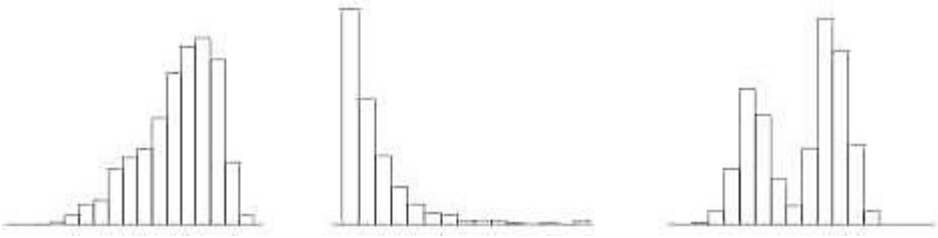
For continuous features we should:

- Examine the mean and standard deviation of each feature to get a sense of the centraltendency and variation of the values within th dataset for the feature.
- Examine the minimum and maximum values to understand the range that is possiblefor each feature.



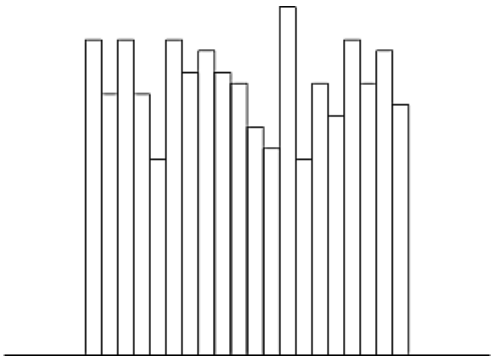
(a) Uniform **(b) Normal (Unimodal)** **(c) Unimodal (skewed right)**

When we generate histograms of features there are a number of common,well understood shapes that we should look out for.



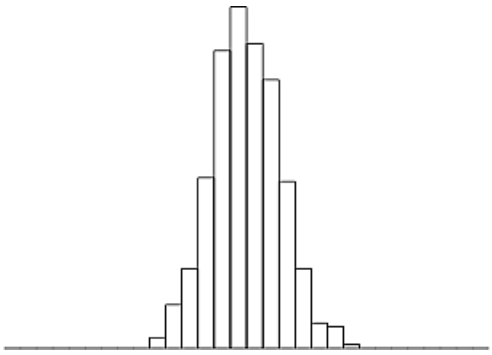
(a) Unimodal (skewed Left) (b) Exponential (b) Multimodal

Figure 4.22 Histograms for different sets of data each of which exhibitwell-known, common characteristics



Uniform

A uniform distribution indicates that a feature isequally likely to take a value in any of theranges present.



Normal (Unimodal)

Features following a normal distribution are characterized by a strong tendency towards a central value and symmetrical variation to either side of this.

The probability density function for the normal distribution (or Gaussian distribution) is

$$N(x, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where x is any value, and μ and σ are parameters that define the shape of the distribution: the population mean and population standard deviation.

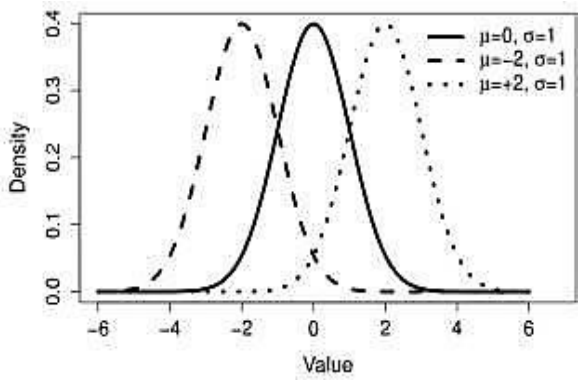


Figure 4.23 Three normal distributions withdifferent means but identical standard deviations.

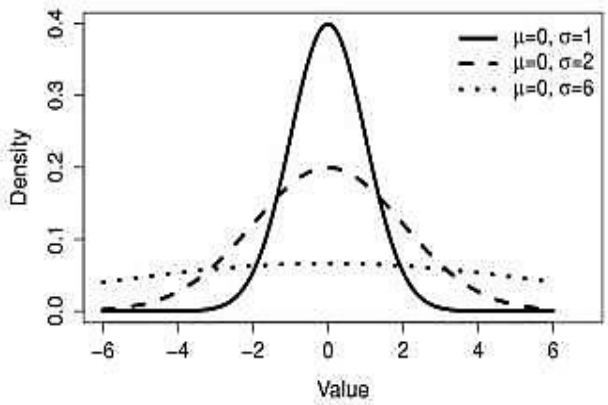


Figure 4.24 Three normal distributions withidentical means but different standard deviations.

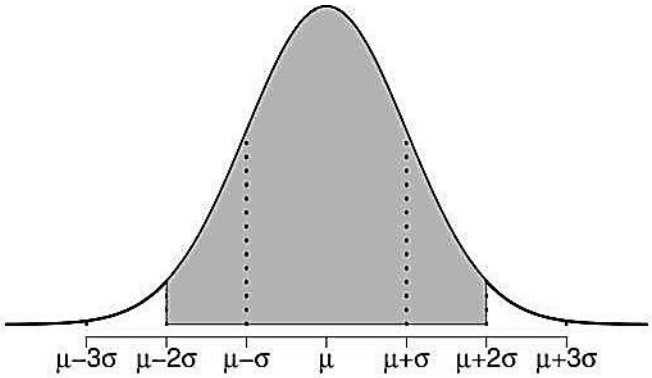


Figure 4.25

Figure An illustration of the 68 95 99:7 percentage rule that a normal distribution defines as the expected distribution of observations. The grey region defines the area where95% of observations are expected.

Identifying Data Quality Issues

A data quality issue is loosely defined as anything unusual about the data in an ABT.The most common data quality issues are:

- missing values
- irregular cardinality
- Outliers

The data quality issues we identify from a data quality report will be of two types:

- Data quality issues due to invalid data
- Data quality issues due to valid data.

Feature	Data Quality Issue	Potential Handling Strategies
NUM SOFT TISSUE	Missing values (2%)	
CLAIM AMOUNT	Outliers (high)	
AMOUNT RECEIVED	Outliers (high)	

Table 4.12 The data quality plan for the motor insurance fraud prediction ABT.

Handling Data Quality Issues

- Approach 1: Drop any features that have missing value.
- Approach 2: Apply complete case analysis.

- Approach 3: Derive a missing indicator feature from features with missingvalue.
- **Imputation** replaces missing feature values with a plausible estimated valuebased on the feature values that are present.
- The most common approach to imputation is to replace missing values for a feature with a measure of the central tendency of that feature
- We would be reluctant to use imputation on features missing in excess of 30% oftheir values and would strongly recommend against the use of imputation on featuresmissing in excess of 50% of their values.
- The easiest way to handle outliers is to use a clamp the transformation that clamps allvalues above an upper threshold and below a lower threshold to this threshold values,thus removing the offending outliers.

where a_i, is a specific value of feature a, and lower and upper are the lower and upperthresholds.

$$a_i = \begin{cases} lower & \text{if } a_i < lower \\ upper & \text{if } a_i > upper \\ a_i & otherwise \end{cases}$$

Case Study: Motor Insurance Fraud

Table 4.13 The data quality plan for the motor insurance fraud prediction ABT.

Feature	Data Quality Issue	Potential Handling Strategies
NUM SOFT TISSUE	Missing values (2%)	Imputation (median: 0.0)
CLAIM AMOUNT	Outliers (high)	Clamp transformation (manual: 0, 80 000)
AMOUNT RECEIVED	Outliers (high)	Clamp transformation (manual: 0, 80 000)

INFORMATION BASED LEARNING

(a) Brian

(b) John

(c) Aphra

(d) Aoife

Figure 4.27 The different question sequences that can follow in a game of Guess-Who beginning with the question Is it a man?

All the paths in this diagram are two questions long.

So, on average if you ask Question (1) first the average number of questions you have to ask per game is:

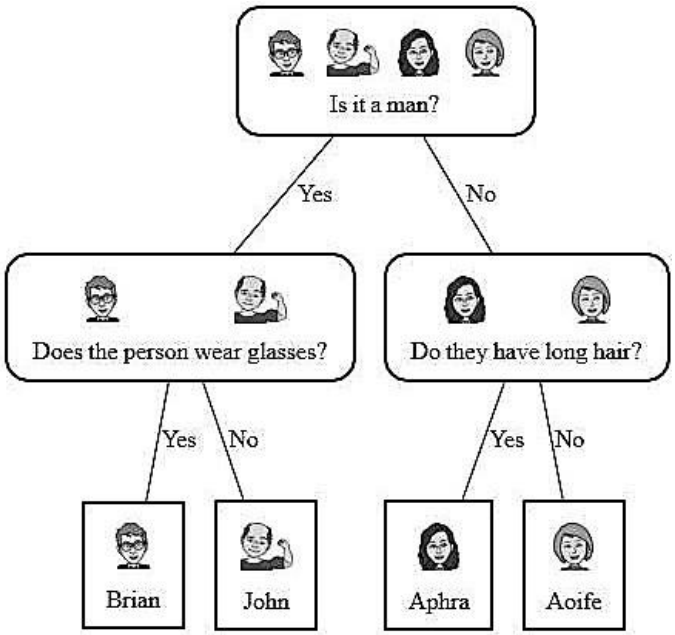


Figure 4.27

- On average getting an answer to Question (1) seems to give you moreinformation than an answer to Question (2): less follow up questions.
- This is not because of the literal message of the answers: YES or NO.

Big Idea

So the big idea here is to figure out which features are the most informative ones to ask questions about by considering the effects of the different answersto the questions, in terms of:

- How the domain is split up after the answer is received,
- Likelihood of each of the answers.

Fundamentals

A decision tree consists of:

- a **root node** (or starting node),

- **interior nodes**
- and **leaf nodes** (or terminating nodes).
- each of the non-leaf nodes (root and interior) in the tree specifies a test to becarried out on one of the query's descriptive features.
- each of the leaf nodes specifies a predicted classification for the query.

Table 4.14 An email spam prediction dataset

ID	SUSPICIOUS WORDS	UNKNOWN SENDER	CONTAINS IMAGES	CLASS
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

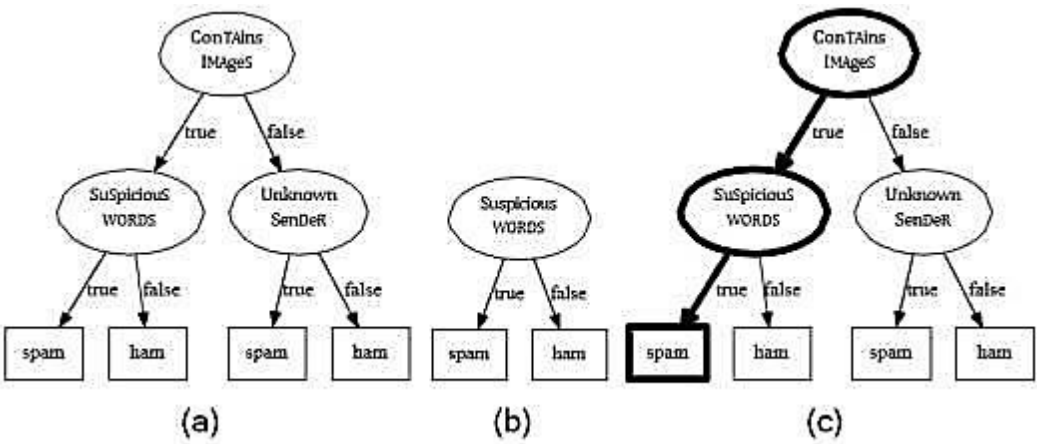


Figure 4.28

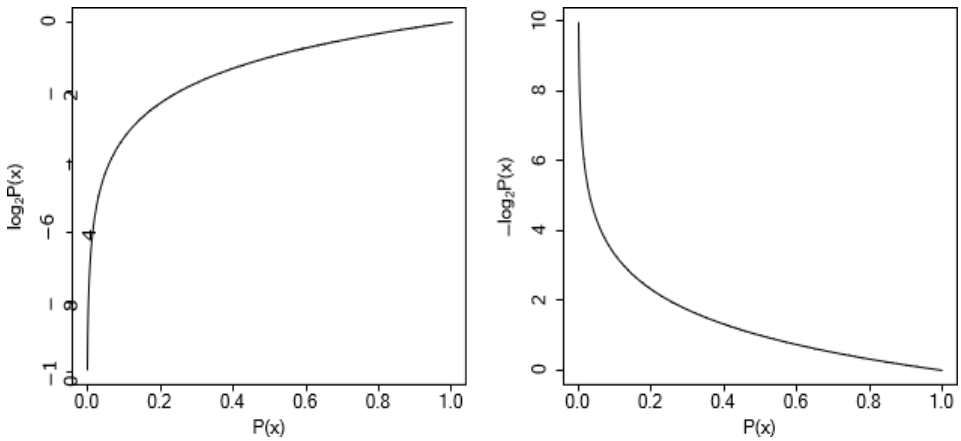
Shannon's Entropy Model

- Claude Shannon's entropy model defines a computational measure of the impurity ofthe elements of a set.
- An easy way to understand the entropy of a set is to think in terms of the uncertaintyassociated with guessing the result if you were to make a random selection from the set.
- Entropy is related to the probability of a outcome.
- High probability → Low entropy

- Low probability → High entropy
- If we take the log of a probability and multiply it by -1 we get this mapping!

  $log_5(25) = 2$ because $5^2 = 25$

  $log_5(32) = 2.153$ because $5^{2.153} = 32$



(a) **(b)**

Figure 4.29 (a) A graph illustrating how the value of a binary log (the log to the base 2) of a probability changes across the range of probability values. (b) the impact of multiplying these values by −1.

Shannon's model of entropy is a weighted sum of the logs of the probabilities of each of thepossible outcomes when we make a random selection from a set.

$$H(t) = -\sum_{i=1}^{l}(P(t = i) \times logs\ (P(t = i))$$

What is the entropy of a set of 52 different playing cards?

$$H(card) = -\sum_{i=1}^{52}(P(card = i) \times logs\ (P(card = i))$$

$$= -\sum_{i=1}^{52} 0.019 \times \log_2(0.019)$$

$$= -\sum_{i=1}^{52} -0.1096$$

$$= -5.700\ bits$$

What is the entropy of a set of 52 playing cards if we only distinguish between the cards based on their suit {♥,♣,♦,♠}?

$$H(suit) = -\sum_{lg\{\heartsuit,\clubsuit,\diamondsuit,\spadesuit\}} (P(suit = l) \times \log_2(P(suit = l))$$

$$= -((P(\heartsuit) \times \log_2(P(\heartsuit))) + (P(\clubsuit) \times \log_2(P(\clubsuit)))$$

$$+ (P(\diamondsuit) \times \log_2(P(\diamondsuit))) + (P(\spadesuit) \times \log_2(P(\spadesuit))))$$

$$= -((\frac{13}{52} \times \log_2 (P (\frac{13}{52}))) + (P (\frac{13}{52}) \times \log_2 (P (\frac{13}{52})))$$

$$+ (P (\frac{13}{52}) \times \log_2 (P (\frac{13}{52})))$$

$$+ (P (\frac{13}{52}) \times \log_2 (P (\frac{13}{52}))))$$

$$= -((0.25 \times -2) + (0.25 \times -2) + (0.25 \times -2) + (0.25 \times -2))$$

$$= 2\ bits$$

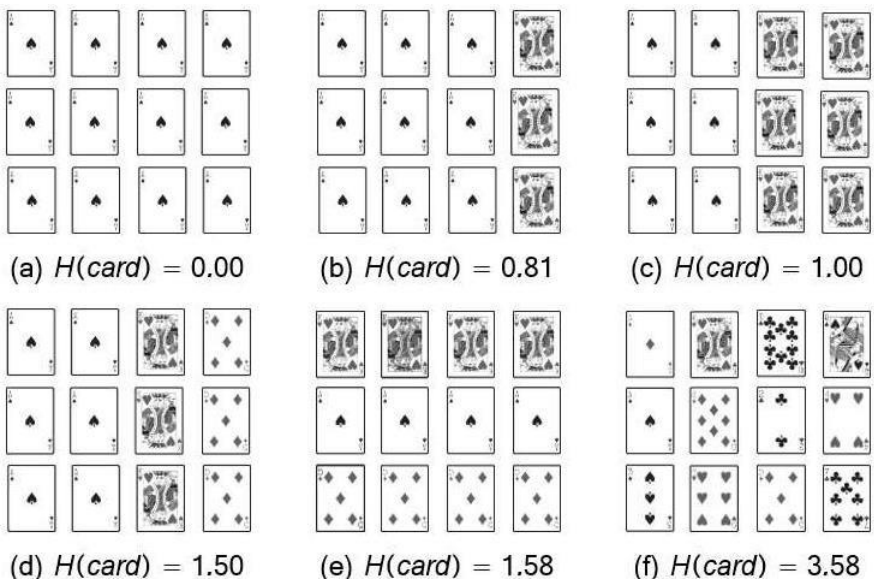


Figure 4.30 The entropy of different sets of playing cards measured in bits.

Entropy of a Message	Properties of the Message Set
High	A large set of equally likely messages.
Medium	A large set of messages, some more likely than others.
Medium	A small set of equally likely messages.
Low	A small set of messages with one very likely message.

Table 4.15 The relationship between the entropy of a message and the set it was selected from

Information Gain

1. Compute the entropy of the original dataset with respect to the target feature. This gives us a measure of how much information is required in order to organize the dataset into pure sets.
2. For each descriptive feature, create the sets that result by partitioning the instances in the dataset using their feature values, and then sum the entropy scores of each of these sets. This gives a measure of the information that remains required to organize the instances into pure sets after we have split them using the descriptivefeature.
3. Subtract the remaining entropy value (computed in step 2) from the original entropy value(computedinstep1) to give the information gain.

The equation calculates the entropy for a dataset with respect to a target feature

$$H(t, D) = -\sum_{lglevels(t)} (P(t = l) \times \log_2(P(t = l))$$

$$rem(d, D) = \sum_{lglevels(t)} \underbrace{\frac{|D_{d=l}|}{|D|}}_{weighting} \times \underbrace{H(t, D_{d=l})}_{\substack{entropy\ of \\ partition\ D_{d=l}}}$$

$$IG(d, D) = H(t, D) - rem(d, D)$$

SIMILARITY-BASED LEARNING

Feature space: each descriptive has its own dimensional axis. It is an abstract m-dimensional space that is created by making each descriptive feature in a dataset an axis of an m-dimensional coordinate system and mapping each instance inthe dataset to a point in this coordinate space based on the values of its descriptive features

Metric(a,b): is a real value function that returns the distance betweentwo points a and b in the feature space. It has following properties:

- Non-negativity
- Identity
- Symmetry
- Triangular inequality
- Two examples of distance metric
- Euclidean distance
- Manhattan distance (taxi-cam distance)=sum of absolute differences.

Minkowski distance: a family of distance metrics based on differences between features.

Nearest Neighbor algorithm:

Algorithm : Nearest neighbor algorithm. Require:

1. Iterate across the instances in memory to find the nearest neighbor—this is the instancewith the shortest distance across the feature space to the query instance.
2. Make a prediction for the query instance that is equal to the value of the target feature ofthe nearest neighbor.

- Decision boundary: is the boundary between regions of the feature space in which different target lavels will be predicted. It is generated by aggregating the neighboring local models (Voronoi regions) that make the same prediction
- **Noise effects:**
- Using Kronecker delta approach:
- **n-n algorithm** is sensitive to noise because any errors in the description of labeling of training data results in erroneous local models and incorrect predictions. One way to mitigate against noise is to modify the algorithm to return the majority target levelwithin the set of k nearest neighbors to the query q

- **Using weighted k nearest neighbor approach:**
- **Efficient Memory search**:
- Assuming that the training dataset will remain relatively stable, the time issue can eboffset by investing in one-off computation to create an index of the instances that enables efficient retrieval of the nearest neighbors without doing an exhaustive search of the entire training dataset.
- **k-d tree:** stands for k-dimensional tree which is a balanced binary tree in which eachof the nodes in the tree index one on the instances in a training dataset
- This tree is constructed so that the nodes that are nearby in the tree index training instances that are nearby in the feature space

PROBABILITY-BASED LEARNING

- We can use estimates of likelihoods to determine the most likely prediction that should be made. More importantly, we revise these predictions based on data we collect and whenever extra evidence becomes available.
- A **probability function**, P(), returns the probability of a feature taking a specific value.
- A **joint probability** refers to the probability of an assignment of specific values to multiple different features
- **Baye's Theorem**

  Bayes' Theorem defines the conditional probability of an event, X, given some evidence, Y, in terms of the product of the inverse conditional probability, P(Y| X), and the prior probability of the event P(X).

$$P\left(\frac{X}{Y}\right) = \frac{P\left(\frac{Y}{X}\right) P(X)}{P(Y)}$$

- **Bayesian Prediction**

  To make Bayesian predictions, we generate the probability of the event that a target feature, t, takes a specific level, l, given the assignment of values to a set of descriptive features, q, from a query instance.

  The Generalized Bayes' Theorem is defined as

$$P(t = l \mid q[1], \dots, q[m]) = \frac{P(q[1], \dots, q[m] \mid t = l) P(t = l)}{P(q[1], \dots, q[m])}$$

To calculate a probability using the Generalized Bayes' Theorem, we need to calculate threeprobabilities:

1. P(t=l),the prior probability of the target feature t taking the level *l*
2. P(q[1],…,q[m]),the joint probability of the descriptive features of a query instance Taking a specific set of values
3. P(q[1], …, q[m] | t = l), the conditional probability of the descriptive features of a query instance taking a specific set of values given that the target feature takes the Level *l*

Conditional Independence and Factorization

If knowledge of one event has no effect on the probabilityof another event, and vice versa, then the two events are independent of each other.

f two events X and Y are independent then:

$$P(X|Y) = P(X)$$

$$P(X, Y) = P(X) \times P(Y)$$

- Full independence between events is quite rare.

ERROR-BASED LEARNING

In error-based machine learning, we perform a search for a set of parameters for a parameterized model that minimizes the total error across the predictions made by that model with respect to a set of training instances.

Simple Linear Regression

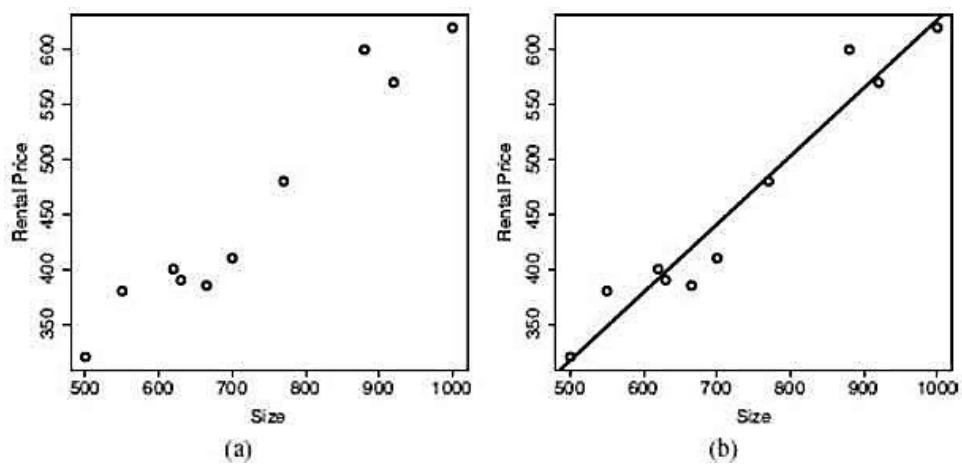


Figure 4.31 a) A scatter plot of the SIZE and RENTAL PRICE features from theoffice

rentals dataset; (b)the scatter plot from (a)with al inear model relating RENTALPRICE toSIZE overlaid.

Measuring Error

In order to formally measure the fit of a linear regression model with a setof training data, we require an error function. An error function captures theerror between the predictions made by a model and the actual values in a training dataset.

- There are many different kinds of error functions, but for measuring the fit of simple linear regression models, the most commonly used is the **sum of squared errors errorfunction, or *L2*.**
- To calculateL2 we use our candidate model to make a prediction for each member of thetraining dataset and then calculate the error (or residual) between these predictions and the actual target feature values in the training set.

Error Surface:

For every possible combination of weights, w[0] and w[1], there is a corresponding sum Of squared errors value.

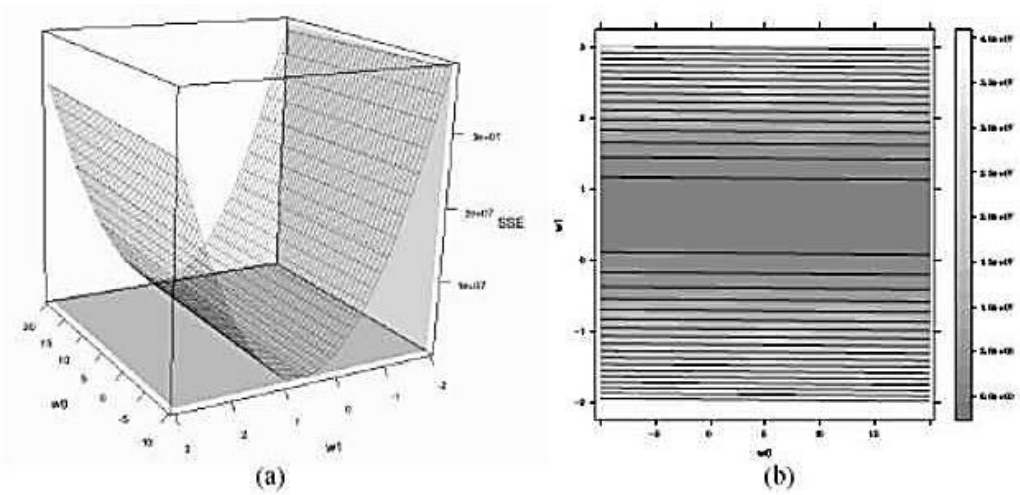


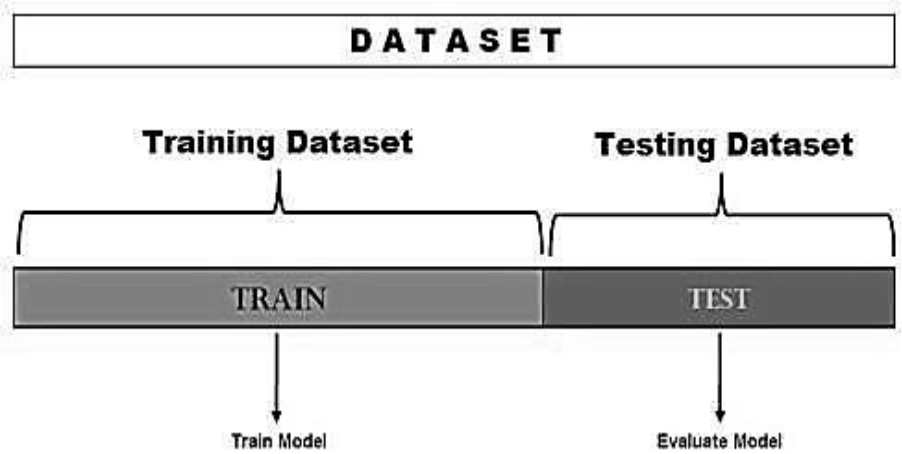
Figure 4.32

EVALUATION

When evaluating machine learning (ML) models, the question that arises is whether the model is the best model available from the model's hypothesis space in terms of generalization error on the unseen / future data set.

Hold-out method for Model Evaluation

The hold-out method for model evaluation represents the mechanism of splitting the dataset into training and test datasets. The model is trained on the training set and then tested on the testing set to get the most optimal model.



In the above diagram, you may note that the data set is split into two parts. One split is set aside or held out for training the model. Another set is set aside or held out for testing or evaluating the model.

THE ART OF MACHINE LEARNING TO PREDICTIVE DATA ANALYTICS.

The CRoss Industry Standard Process for Data Mining (CRISP-DM) is a process model thatserves as the base for a data science process. It has six sequential phases:

1. Business understanding – What does the business need?
2. Data understanding – What data do we have / need? Is it clean?
3. Data preparation – How do we organize the data for modeling?
4. Modeling – What modeling techniques should we apply?
5. Evaluation – Which model best meets the business objectives?
6. Deployment – How do stakeholders access the results?

UNIT V

APPLICATIONS OF MACHINE LEARNING

Image Recognition - Speech Recognition - Email spam and Malware Filtering - Online Fraud detection - Medical Diagnosis.

IMAGE RECOGNITION

A popular dataset that we haven't talked much about yet is the Olivetti face dataset. The Olivetti face dataset was collected in 1990 by AT&T Laboratories Cambridge.Thedatasetcomprisesfacialimagesof40distinctsubjects,takenatdifferent times and under different lighting conditions. In addition, subjects varied their facial expression (open/closed eyes, smiling/not smiling) and their facial details (glasses/no glasses).

Images were then quantized to 256 grayscale levels and stored as unsigned 8-bit integers. Because there are40 distinct subjects, the dataset comes with 40 distinct target labels. Recognizing faces thus constitutes an example of a multiclass classification task.

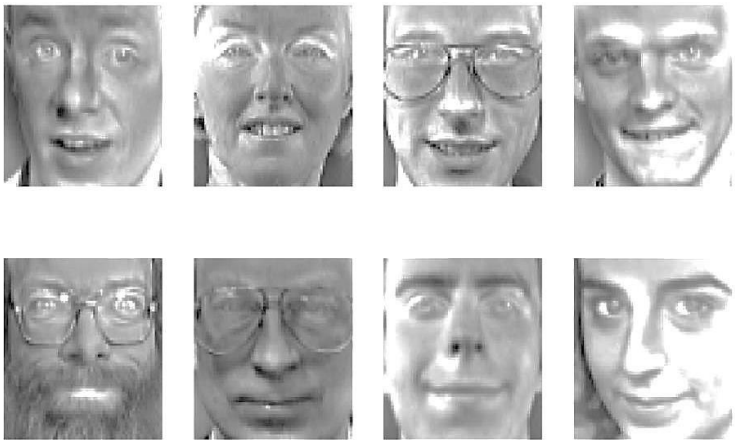
Loading the dataset:

```
In [3]:

import numpy as np
np.random.seed(21)
idx_rand = np.random.randint(len(X), size=8)
```

WecanplottheseexampleimagesusingMatplotlib,butweneedtomakesurewe reshape the column vectors to 64 x 64 pixel images before plotting:

```
In [4]:

import matplotlib.pyplot as plt
%matplotlib inline
plt.figure(figsize=(14, 8))
for p, i in enumerate(idx_rand):
    plt.subplot(2, 4, p + 1)
    plt.imshow(X[i, :].reshape((64, 64)), cmap='gray')
    plt.axis('off')
```

Preprocessing the dataset



Training and testing the random forest

```
In [12]:

rtree.setMinSampleCount(2)
```

However, we might not want to limit the depth of each tree. This is again, a parameter we will have to experiment with in the end. But for now, let's set it to a large integer value, making thedepth effectively unconstrained:

```
In [13]:
In [14]:

rtree.train(X_train, cv2.ml.ROW_SAMPLE, y_train);
```

This means that although we allowed the tree to go up to depth 1000, in the end only 25 layers were needed.

The evaluation of the classifier is done once again by predicting the labels first

(y_hat) and thenpassing them to the accuracy_score function:

```
In [16]:

_, y_hat = rtree.predict(X_test)
```

```
In [17]:

from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_hat)
```

We find 87% accuracy, which turns out to be much better than with a single decision tree:

```
In [18]:

from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier(random_state=21, max_depth=25)
tree.fit(X_train, y_train)
tree.score(X_test, y_test)

Out[18]:

0.46999999999999997
```

We can play with the optional parameters to see if we get better. The most importantoneseemstobethenumberoftreesintheforest.Wecanrepeattheexperiment with a forestmade from 100 trees:

```
In [19]:

num_trees = 100
eps = 0.01
criteria = (cv2.TERM_CRITERIA_MAX_ITER + cv2.TERM_CRITERIA_EPS,
            num_trees, eps)
rtree.setTermCriteria(criteria)
rtree.train(X_train, cv2.ml.ROW_SAMPLE, y_train);
_, y_hat = rtree.predict(X_test)
accuracy_score(y_test, y_hat)

Out[19]:

0.91000000000000003
```

SPEECH RECOGNITION

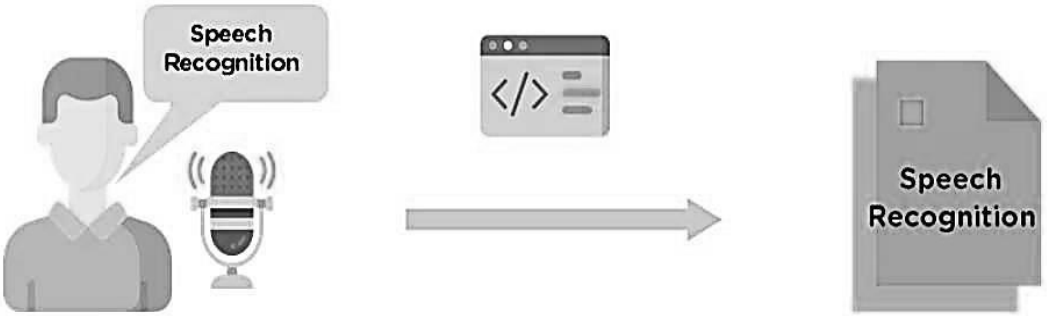
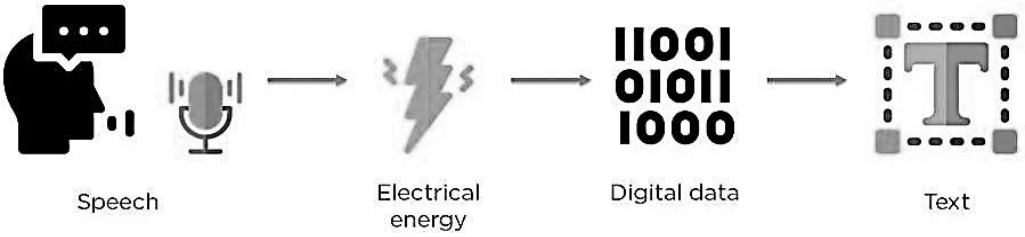


Figure Speech Recognition

Speech recognition seems highly futuristic, but it is present all around you. Automated phone calls allow you to speak out your query or the query you wish to be assisted on; your virtual assistants like Siri or Alexa also use speech recognition to talk to you seamlessly.

How Does Speech Recognitionwork?

Speech recognition in Python works with algorithms that perform linguistic and acoustic modeling. Acoustic modeling is used to recognize phenones/phonetics in our speech to get the more significant part of speech, as words and sentences.



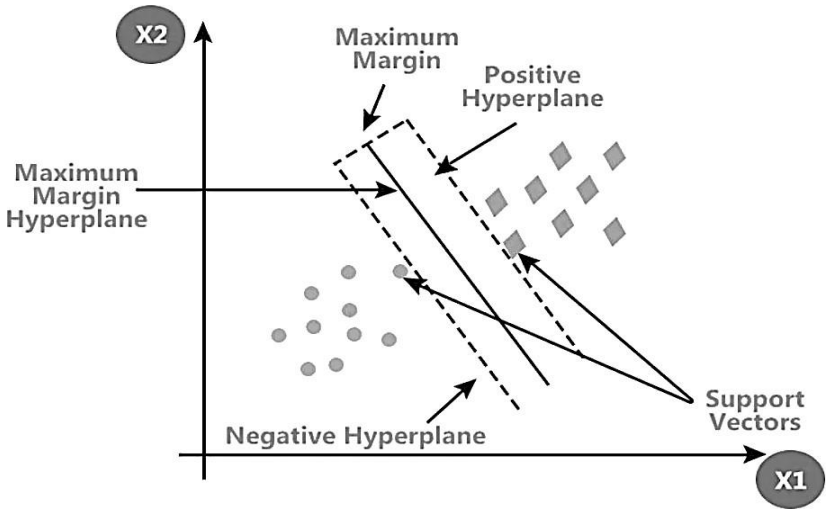
SPAMFILTERING

Spam Detector is used to detect unwanted, malicious and virus infected texts and helps to separate them from the non spam texts. It uses a binary type of classification containingthelabelssuchas‘ham’(nonspam)andspam.Applicationofthiscanbeseen in Google Mail (GMAIL) where it segregates the spam emails in order to prevent them from getting into the user’s inbox.

In this Machine Learning Spam Filtering application, we will develop a Spam Detector app using Support Vector Machine (SVM) technique for classification and Natural Language Processing. We will detect whether the piece of input text is “ham”(nonspam) or “spam”. We will split our dataset into training and testing and then train our classifier with SVM classifier.

What is a Support Vector Machine?

Support Vector Machine (SVM) is a supervised learning algorithm used for classificationandregressionproblems.ThemainobjectiveofSVMistofindahyperplane inanN(total number of features)-dimensional space that differentiates the data points. So we need to find a plane that creates the maximum margin between two data point classes.



.

Prerequisites:

This project requires you to have a good knowledge of Python and Natural Language Processing(NLP).Modulesrequiredforthisprojectarepandas,pickle,sklearn, numpy and nltk.You can install with using following command:

```
1.  pip install pandas , pickle , sklearn , numpy , nltk
```

The versions which are used in this application for pythonand its corresponding modules are as follows:

1. python: 3.8.5
2. sklearn: 0.24.2
3. pickle: 4.0
4. numpy : 1.19.5
5. pandas: 1.1.56)nltk : 3.2.5

Steps for developing a Spam Detector:

1) Import Libraries and initialize variables.

Firstly create a file called“spam detector.py”and import all libraries which have been shared in the prerequisites section.

```
#import all the modules
import nltk
nltk.download("punkt")
from nltk.tokenize import word_tokenize
from nltk.stem import LancasterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split as ttsplit
from sklearn import svm
import pandas as pd
import pickle
import numpy as np
```

2) Preprocessing the data

Inthis,wewillusepython’spandas’moduletoreadthedatasetfilewhichweare using for training and testing purposes. Then we will use the “message_X” variable to storefeatures(EmailTextcolumn)and“labels_Y”variabletostoretarget(Labelcolumn) from our dataset.

```
#read the dataset file
df = pd.read_csv("spam.csv")
message_X = df.iloc[:,1] #EmailText column
labels_Y = df.iloc[:,0]  #Label
```

After we get the

Dataset file:

	Label	EmailText
2	ham	Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
3	ham	Ok lar... Joking wif u oni...
4	spam	Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 084
5	ham	U dun say so early hor... U c already then say...
6	ham	Nah I don't think he goes to usf, he lives around here though
7	spam	FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, ÃƒÂ¥
8	ham	Even my brother is not like to speak with me. They treat me like aids patent.
9	ham	As per your request 'Melle Melle (Oru Minnaminunginte Nurungu Vettam)' has been set as your callertune for all Callers. Press *9 to copy y
10	spam	WINNER!! As a valued network customer you have been selected to receivea ÃƒÂ¥Ã‚Â£900 prize reward! To claim call 09061701461. Cla
11	spam	Had your mobile 11 months or more? U R entitled to Update to the latest colour mobiles with camera for Free! Call The Mobile Update Co F
12	ham	I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today.
13	spam	SIX chances to win CASH! From 100 to 20,000 pounds txt> CSH11 and send to 87575. Cost 150p/day, 6days, 16+ TsandCs apply Reply I
14	spam	URGENT! You have won a 1 week FREE membership in our ÃƒÂ¥Ã‚Â£100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010 T&C ww
15	ham	I've been searching for the right words to thank you for this breather. I promise i wont take your help for granted and will fulfil my promise. Yo
16	ham	I HAVE A DATE ON SUNDAY WITH WILL!!

```
#stemming variable initialization
lstem = LancasterStemmer()
def mess(messages):
  message_x = []
  for me_x in messages:
    #filter out other datas except alphabets
    me_x=''.join(filter(lambda mes:(mes.isalpha() or mes==" ") ,me_x))
    #tokenize or split the messages into respective words
    words = word_tokenize(me_x)
    #stem the words to their root words
    message_x+=[' '.join([lstem.stem(word) for word in words])]
  return message_x
```

```
message_x = mess(message_X)
#vectorization process for Machine learning Spam Filtering project
#ignore stop words i.e. words that are of least importance
tfvec=TfidfVectorizer(stop_words='english')
#vectorizing feature data
x_new=tfvec.fit_transform(message_x).toarray()
```

```
#replace ham and spam label with 0 and 1 respectively
y_new = np.array(labels_Y.replace(to_replace=['ham', 'spam'], value=[0, 1]))
```

3) Training the model

```
#split our dataset into training and testing part
x_train , x_test , y_train , y_test = ttsplit(x_new,y_new,test_size=0.2,shuffle=True)
#use svm classifier to fit our model for training process
classifier = svm.SVC()
classifier.fit(x_train,y_train)
```

After training the classifier we will get the following information about best-fit parameters of SVM for our dataset.

Kernel: The kernel is decided on the basis of data transformation. By default, the kernel is the Radial Basis Function kernel(RBF).We Can change it to linear or Polynomial depending on our dataset.

Gamma: Gamma is a nonlinear hyperplane parameter. High values indicate that data points that are very close to each other can be grouped. A low value indicates that data points can be grouped together even if they are separated by large distances.

```
SVC(C=1.0, break_ties=False, cache_size=200, class_weight=None, coef0=0.0,
```

```
#store the classifier as well as messages feature for prediction
pickle.dump({'classifier':classifier,'message_x':message_x},open("training_data.pkl","wb"))
```

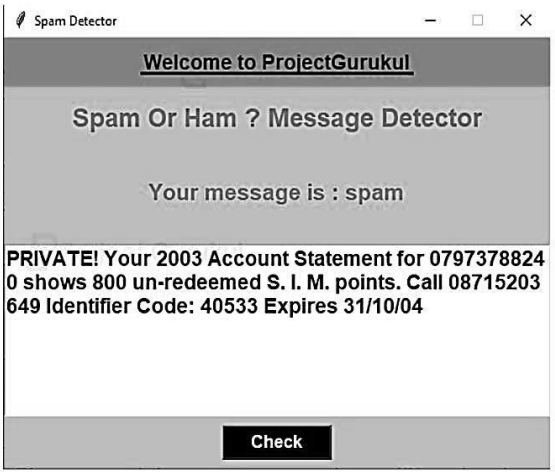
4) Prediction using Graphical User Interface

```
#import all the modules
from tkinter import *
import nltk
from nltk.tokenize import word_tokenize
from nltk.sten import LancasterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
import pickle

BG_COLOR="#99CFF0"
FONT_BOLD="Helvetica %d bold"

class SpamBox:
    def __init__(self):
        #initialize tkinter window
        self.window=Tk()
        self.main_window()
        self.lstem = LancasterStemmer()
        self.tfvec=TfidfVectorizer(stop_words='english')
        self.datafile()

    def datafile(self):
        #get all data from datafile and load the classifier.
        datafile = pickle.load(open("training_data.pkl","rb"))
        self.message_x = datafile["message_x"]
        self.classifier = datafile["classifier"]
```

```
def main_window(self):
        #add title to window and configure it
        self.window.title("Spam Detector")
        self.window.resizable(width=False,height=False)
        self.window.configure(width=520,height=400,bg=BG_COLOR)

        #head label for the window heading
        head_label=Label(self.window,bg="#FFA500",fg="#000",text="Welcome to
        ProjectSurukul", font=FONT_BOLD%(14),pady=10)
        head_label.place(relwidth=1)
        line = Label(self.window,width=200,bg="#000")
        line.place(relwidth=0.5,relx=0.25,rely=0.08,relheight=0.008)

        #mid_label
        mid_label=Label(self.window,bg=BG_COLOR,fg="#0000FF",text="Spam Or
        Ham ? Message Detector", font=FONT_BOLD%(18),pady=10)
        mid_label.place(relwidth=1,rely=0.12)

        #answer label where our prediction about user input message will be displayed
        self.answer=Label(self.window,bg=BG_COLOR,fg="#000",text="Please type
        message below.", font=FONT_BOLD%(16),pady=10,wraplength=525)
        self.answer.place(relwidth=1,rely=0.30)

        #textbox for user to write msg for checking
        self.msg_entry=Text(self.window,bg="#FFF",
        fg="#000",font=FONT_BOLD%(14))
        self.msg_entry.place(relwidth=1,relheight=0.4,rely=0.43)
        self.msg_entry.focus()

        #check button to call the prediction function
        check_button=Button(self.window,text="Check",
                font=FONT_BOLD%(12),width=8,bg="#000",fg="#FFF",
                        command=lambda: self.on_enter(None))
        check_button.place(relx=0.40,rely=0.90,relheight=0.08,relwidth=0.20)
```

.

Machine Learning Spam Filtering Output



CREDIT CARD FRAUD DETECTION

Nowadays most people prefer to do payments by cards and don't like to carry cash with them. That leads to an increase in the use of cards and also thereby frauds.

Credit card frauds are easy to do, as we know that E-commerce and many otheronlinesiteshaveincreasedtheonlinepaymentmodes,Whichaffectsincreasingthe risk of online frauds.

Dataset

In credit card fraud detection project, we will use the dataset which is a csv file. The data set consists of transactions that occurred in two days,where there are 492

1.)Importing Libraries:

Thefirststepofalltheprojectswillbealwaysimporting therequiredlibraries.

```
# import libraries for ProjectGurukul Credit Card Fraud Detection Project using Machine Learning:

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix , accuracy_score, classification_report
```

2) Load the dataset:

```
#Loading the dataset to a Pandas Dataframe

credit_card_data = pd.read_csv('creditcard.csv')
```

3) Analysing and Visualizing the dataset:

In this step we will analyze the dataset and perform certain operations to cleanthe data and make it ready, to train our model. And also we will visualize the dataset.

```
# let's see first 5 rows of the dataset:
credit_card_data.head(5)
```

In [3]:
```
# let's see first 5 rows of the dataset:
credit_card_data.head(5)
```

Out[3]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.1
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.1
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.3
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.6
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.2

5 rows × 31 columns

```
# let's see last 5 rows of our dataset:
credit_card_data.tail()
```

```
# dataset information:
credit_card_data.info()
```

In [5]:
```
# dataset information:
credit_card_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

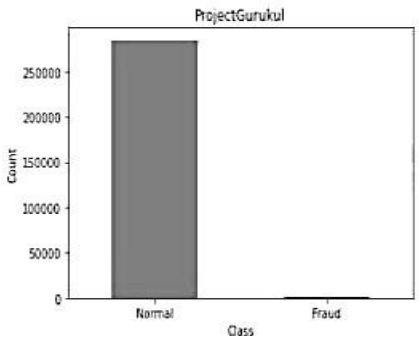
```
# checking number of missing values:
credit_card_data.isnull().sum()

# Find distribution of Normal transaction or Fraud transaction:
credit_card_data['Class'].value_counts()
```

```
In [7]: # Find distribution of Normal transaction or Fraud transaction:

        credit_card_data['Class'].value_counts()

Out[7]: 0    284315
        1       492
        Name: Class, dtype: int64
```

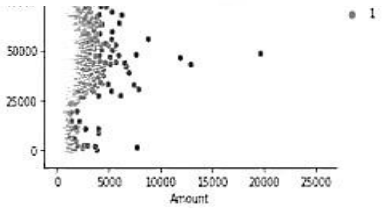
Now we will separate Normal and Fraud transactions, and analyze and visualize that fraud and normal data :

```
# Separating the data:
normal = credit_card_data[credit_card_data.Class == 0]
fraud = credit_card_data[credit_card_data.Class == 1]
# check shape
print(normal.shape)
print(fraud.shape)

#visualize the data:
labels = ["Normal", "Fraud"]
count_classes = credit_card_data.value_counts(credit_card_data['Class'], sort= True)
count_classes.plot(kind = "bar", rot = 0)
plt.title("ProjectGurukul")
plt.ylabel("Count")
plt.xticks(range(2), labels)
plt.show()
```

```
In [12]: #visualize the data:

         labels = ["Normal", "Fraud"]
         count_classes = credit_card_data.value_counts(credit_card_data['Class'], sort= True)
         count_classes.plot(kind = "bar", rot = 0)
         plt.title("ProjectGurukul")
         plt.ylabel("Count")
         plt.xticks(range(2), labels)
         plt.show()
```



```
# Compare values of both transactions:
credit_card_data.groupby('Class').mean()

# Now we will build a sample dataset containing similar distribution of normal transaction and fraud transaction:
normal_sample = normal.sample(n=492)
# Concat two data ( normal_sample and fraud) to create new dataframe which consist equal number of fraud transactions and normal transactions, In this way we balance our dataset (As our dataset is highly unbalanced initially) :
credit_card_new_data = pd.concat([normal_sample, fraud], axis=0)
Let's see our new dataset:
credit_card_new_data

# Analyse our new dataset:
credit_card_new_data['Class'].value_counts()
```



4) Splitting the data:

After analyzing and visualizing our data, now we will split our dataset in X and Y or say in features and labels:

```
# Splitting data into features and targets
X = credit_card_new_data.drop('Class', axis=1)
Y = credit_card_new_data['Class']

# splitting the data into training and testing data:
X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size = 0.2, stratify = Y, random_state= 2)
print(X.shape, X_train.shape, X_test.shape)
```

5.) Creating Logistic Regression Model:

Now we will create the machine learning model.

```
# Creating Model:
model = LogisticRegression()
# training the Logistic Regression model with training data:
model.fit(X_train,Y_train)
```

6.) Model Evaluation:

```
# Model Evaluation
X_train_pred = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_pred, Y_train)
print('Accuracy of Training data:', training_data_accuracy)
```

```
In [30]: # Model Evaluation

         X_train_pred = model.predict(X_train)
         training_data_accuracy = accuracy_score(X_train_pred, Y_train)

         print('Accuracy of Training data:', training_data_accuracy)

         Accuracy of Training data: 0.9584447268106735
```

As you can see the model we have created gives 95% accuracy on training data. The accuracy is very good as we are training our model on very less data. So on considering that our model accuracy is good.

```
# classification report of the model on training data:
print(classification_report(X_train_pred, Y_train))
```

```
# accuracy on test data:
X_test_pred = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_pred, Y_test)
print('Accuracy of Testing data:', test_data_accuracy)

# confusion matrix and classification report of test data:
print(confusion_matrix(X_test_pred, Y_test))
print(classification_report(X_test_pred, Y_test))
```

```
In [31]: # accuracy on test data:

         X_test_pred = model.predict(X_test)
         test_data_accuracy = accuracy_score(X_test_pred, Y_test)

         print('Accuracy of Testing data:', test_data_accuracy)

         Accuracy of Testing data: 0.9289340101522843

In [32]: print(confusion_matrix(X_test_pred, Y_test))

         [[95 10]
          [ 4 88]]

In [34]: print(classification_report(X_test_pred, Y_test))

                      precision    recall  f1-score   support

                   0       0.96      0.90      0.93       105
                   1       0.90      0.96      0.93        92

            accuracy                           0.93       197
           macro avg       0.93      0.93      0.93       197
        weighted avg       0.93      0.93      0.93       197
```

Asyoucan seeour modelaccuracy ontest datais92%

Medical Diagnosis using Machine Learning

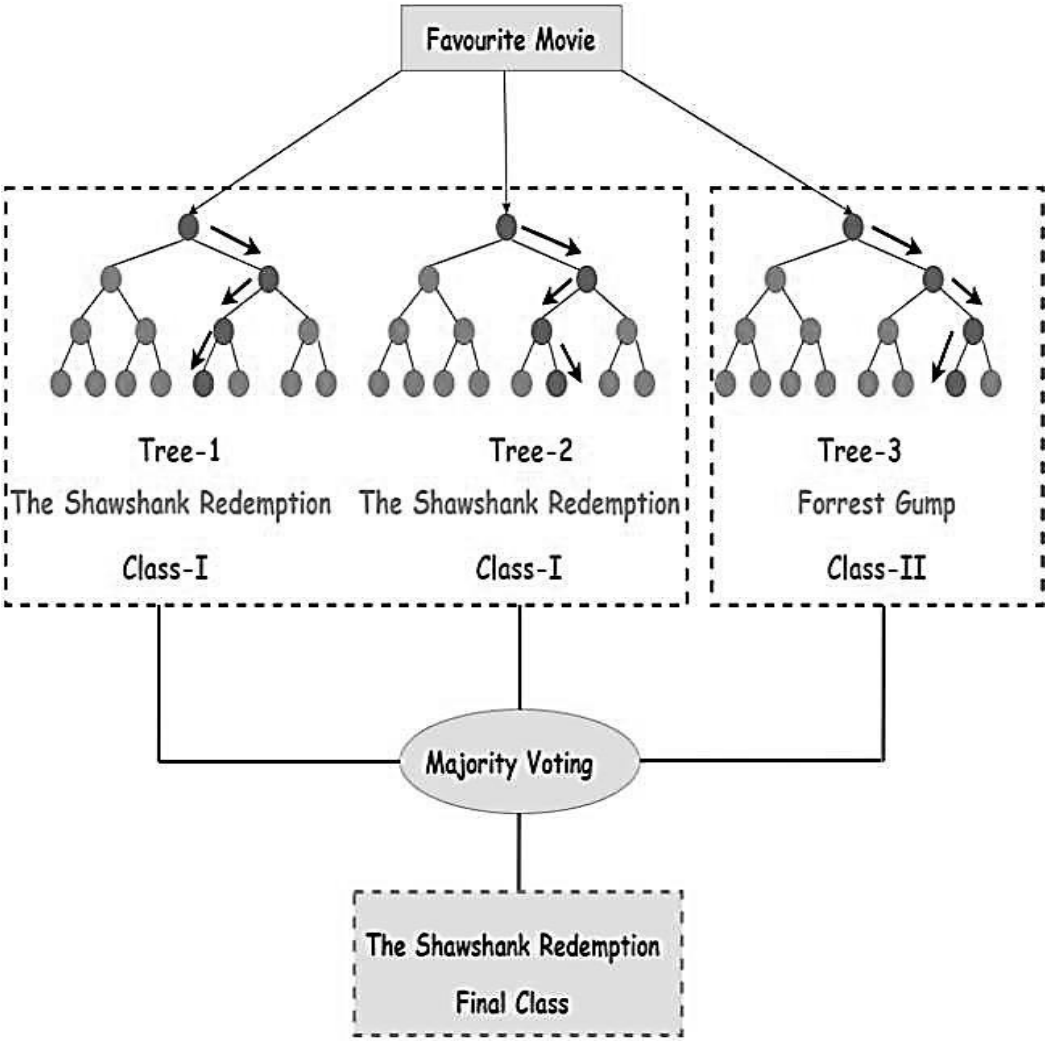
Detecting Parkinson’s Disease:

What is Parkinson’s Disease?

About Detecting Parkinson’s Disease Application:

In this Python machine learning application, we will build a model to detect Parkinson’s disease using one of the Classifier techniques known as Random Forest Classifier as our output contains only 1’s and 0’s. We’ll load the dataset, get the features and targets, split them in to training and testing sets, and finally pass them to Random Forest Classifier for prediction.

Random forest example:



As you can see in the above image we are building a random forest for the topic "Favourite movie". We have used three trees for the classification. Let's say these three trees are three different persons. So after asking different questions and applying the conditions we can see that the first two people said their favorite movie is "The Shawshank Redemption" and one said "Forrest Gump".

As Random forest classifier uses majority voting technique, we get "The Shawshank Redemption"as our Final Class output.

```
#predict the output for x_test
y_pred=model.predict(x_test)

#calculate accuracy,root mean squared error
print('Accuracy :',accuracy_score(y_test, y_pred))
print('Mean Absolute Error:', mean_absolute_error(y_test, y_pred))
print('Root Mean Squared Error:', np.sqrt(mean_squared_error(y_test, y_pred)))
```

Output:

```
= RESTART: X:\Projects\ProjectGurukul\Detecting Parkinson's
Accuracy : 0.9743589743589743
Mean Absolute Error: 0.02564102564102564
Root Mean Squared Error: 0.16012815380508713
```

Initialize‘RandomForestClassifier’andtrainthemodel.

```
#initialize the random forest classifier and fit the dates
model= RandomForestClassifier(random_state=2)
model.fit(x_train,y_train)
```

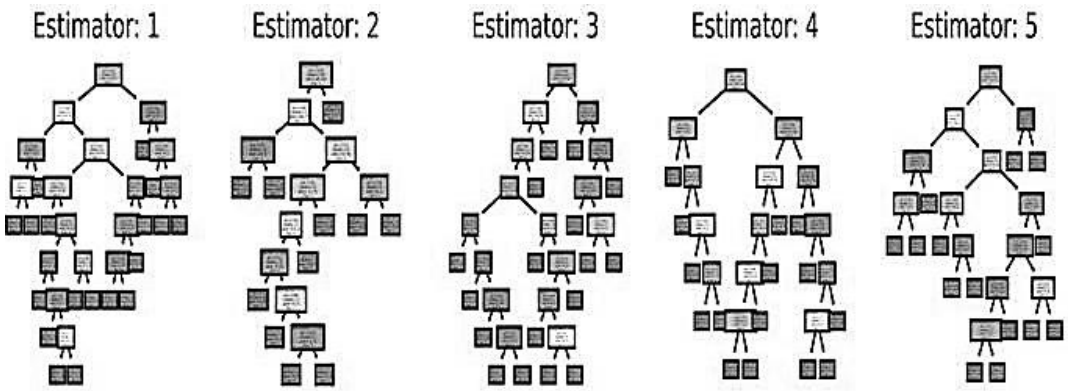
Output:

```
RandomForestClassifier(bootstrap=True, ccp_alpha=0.0, class_weight=None,
                       criterion='gini', max_depth=None, max_features='auto',
                       max_leaf_nodes=None, max_samples=None,
                       min_impurity_decrease=0.0, min_impurity_split=None,
                       min_samples_leaf=1, min_samples_split=2,
                       min_weight_fraction_leaf=0.0, n_estimators=100,
                       n_jobs=None, oob_score=False, random_state=2, verbose=0,
                       warm_start=False)
```

Prediction and Accuracy

Now we will predict our output(y_pred) for testing data(x_test) which is 20% of the dataset using the model which we have trained. Also, we will calculate accuracy, mean absolute error and root mean square error of our model.

1) **Building thePrediction System.**



Finally,wewilltaketheuser'sinputandcheckwhetherdatahasParkinson'sdisease or not.

```
#input data and transform into numpy array
in_data= np.asarray(tuple(map(float,input().rstrip().split(','))))
#reshape and scale the input array
in_data_re = in_data.reshape(1,-1)
in_data_sca = scaler.transform(in_data_re)

#print the predicted output for input array
print("Parkinson's Disease Detected" if model.predict(in_data_sca) else "No Parkinson's Disease Detected")
```

Detect Parkinson's Disease Output

```
Enter the data:
148.46200,161.07800,141.99800,0.00397,0.00003,0.00202,0.00235,0.00605,0.01831,0.163
00,0.00950,0.01103,0.01559,0.02849,0.00639,22.86600,0.408598,0.768845,-5.704053,0.2
16204,2.679185,0.197710
Parkinson's Disease Detected

Enter the data:
115.38000,123.10900,108.63400,0.00332,0.00003,0.00160,0.00199,0.00480,0.01503,0.137
00,0.00812,0.00933,0.01133,0.02436,0.00401,26.00500,0.405991,0.761255,-5.966779,0.1
97938,1.974857,0.184067
No Parkinson's Disease Detected
```